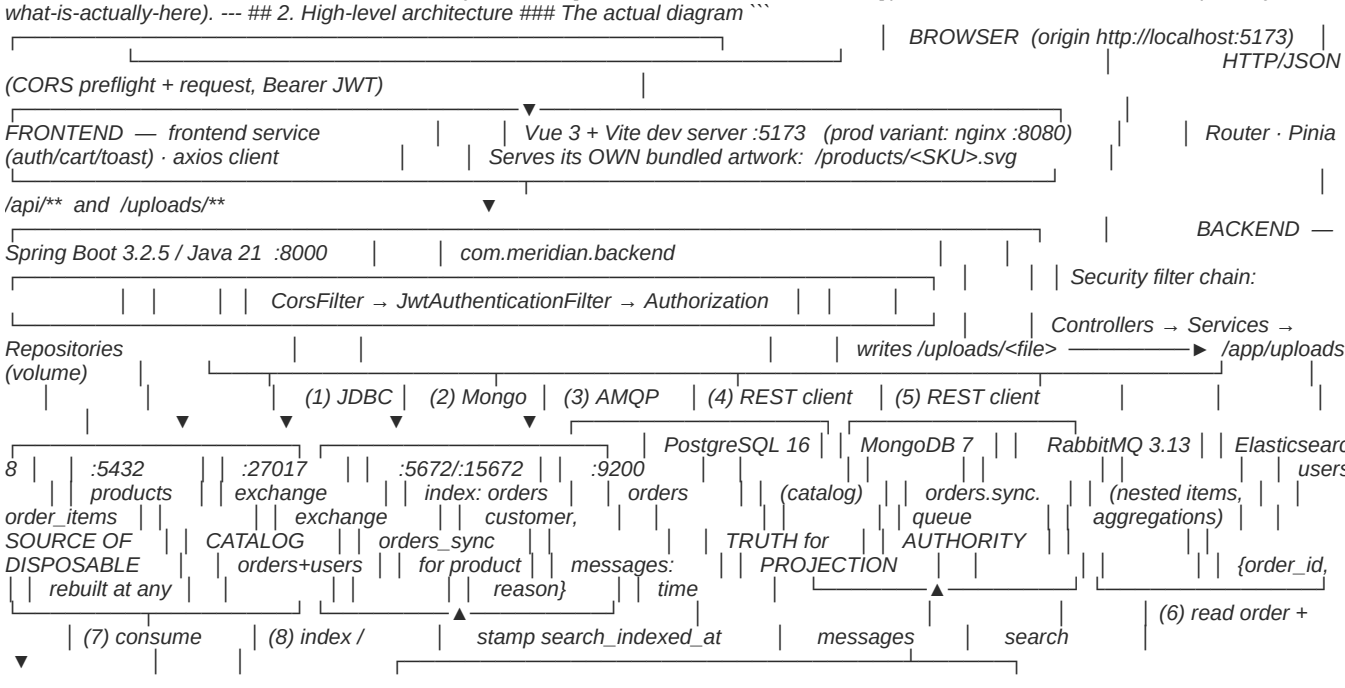
System Architecture & Workflow Summary BUILD_ON: None — starting point IN PLAIN TERMS: This project is an e-commerce demo (Meridian

https://gemini.google.com/app/7320283742643293

*User prompt: we have to make knowlegde bytes for the project to understand the working, workflow, architecture of the project this is the the original prompt for knowledge bytes -> You are acting as a patient senior engineer walking a teammate through an unfamiliar piece of code. Your job is to break the following code document into a series of small, self-contained explanations called "KNOWLEDGE BYTES." RULES FOR KNOWLEDGE BYTES: 1. Each byte covers ONE concept, function, block, or responsibility — never more. 2. Each byte must be understandable on its own, but should note what earlier bytes it builds on (if any). 3. Order the bytes so understanding builds progressively: context/purpose first, then structure, then details, then edgecases/gotchas last. 4. Use plain language first, then show the relevant code snippet, then explain it. 5. Avoid restating the code line-by-line — explain WHAT it does and WHY it exists, not just a transliteration of syntax into English. 6. Keep each byte short enough to read in under 45 seconds. 7. Do not assume prior knowledge of this specific codebase — define any domain-specific terms or patterns the first time they appear. OUTPUT FORMAT (repeat for each byte): --- ### Byte [N]: [Short, descriptive title] **Builds on:** [Byte # or "None — starting point"] **In plain terms:** [1-3 sentences explaining the concept/purpose in everyday language] **The code:** ```[language] [relevant snippet only — not the whole file] modify this prompt to fit the summary of the project system architecture -> # Meridian — System Architecture > **Part A of the system knowledge package.** > Companion documents: [`SYSTEM_WORKFLOW.md`](SYSTEM_WORKFLOW.md) (how a request travels end to end) > and [`FILE_BY_FILE_GUIDE.md`](FILE_BY_FILE_GUIDE.md) (what every source file does). This document explains **what the system is, how it is shaped, and why**. It is written for a developer who knows basic programming but has not seen this project before. Every diagram, class name, endpoint and number below was taken from the actual source code and the running system — nothing is invented. --- ## Table of contents | § | Section | |---|---| | [1](#1-what-the-project-is) | What the project is | | [2](#2-high-level-architecture) | High-level architecture + ASCII diagram | | [3](#3-database-responsibility) | Database responsibility (MongoDB / PostgreSQL / Elasticsearch / RabbitMQ / worker) | | [4](#4-order-snapshot-semantics) | Order snapshot semantics | | [5](#5-search-architecture) | Search architecture | | [6](#6-image-architecture) | Image architecture | | [7](#7-docker-architecture) | Docker architecture | | [8](#8-api-map) | API map (all 16 endpoints) | | [9](#9-important-design-decisions) | Important design decisions | | [10](#10-known-caveats) | Known caveats | --- ## 1. What the project is Meridian is a **demonstration e-commerce order-management and search service**. It models the part of an online shop that customers and staff both touch, split across four backing systems. ### What the customer does 1. Browses the storefront — product grid, category rail, tag marquee, text search, sort and pagination (`StorefrontView.vue`). 2. Adds products to a cart (`stores/cart.js`, opened from `CartDrawer.vue`). 3. Goes to checkout, where prices are **re-fetched live** from the server before placing the order (`CheckoutView.vue`). 4. Places the order — the backend prices it against MongoDB, writes it to PostgreSQL, and publishes a "sync" message to RabbitMQ. 5. Views their own order details, and can see whether it has reached the search index. ### What the admin does 1. Signs in at `/login` with an ADMIN account (`admin@example.com` / `admin`). 2. Lands on the **search dashboard** (`/admin/search`) — Elasticsearch-backed order search with KPI cards (revenue, order counts by status), status/date/price filters, phrase and as-you-type search, and pagination. 3. Opens a single order (`/admin/orders/:id`) — served from **PostgreSQL** as the source of truth — and changes its status (`PENDING` → `PROCESSING` → `SHIPPED`). 4. Manages the catalog (`/admin/catalog`): create, edit, deactivate products and upload product images. ### Why multiple databases? A single relational database *could* store all of this. The project deliberately does not, because the four jobs have genuinely different shapes: | Job | Shape of the data | Why the obvious database fits | |---|---|---| | Product catalog | Deeply nested: `tags[]`, `attributes{}`, `variants[]` per product | MongoDB stores the whole product as one natural document — no JOINs for a nested array that is always read together | | Orders | Rigid, relational, must be transactional and auditable | PostgreSQL gives ACID writes, foreign keys, and stable historical records | | Search | Filter across thousands of orders in milliseconds, plus aggregations | Elasticsearch is purpose-built for inverted indexes and facets | | Keeping search in sync | Search must never block placing an order | RabbitMQ lets the write finish first and sync later | This pattern — **polyglot persistence** — is the point of the exercise: use each store for the thing it is best at, and define the boundary between them precisely. ### What polyglot persistence solves here * **Writes stay fast and correct.** Placing an order commits once in PostgreSQL. Indexing into Elasticsearch happens afterwards, out of band. A slow search cluster cannot make checkout fail. * **Reads are shaped for the reader.** The storefront reads a *product* (one document). The admin dashboard reads *orders filtered and aggregated* (an index). The order detail page reads *one authoritative order row*. Each store answers in its own dialect. * **Failure is isolated.** If Elasticsearch is down, browsing and checkout still work; only `/api/search/orders` reports `503 search_unavailable`. `/api/health` reports exactly which of the four dependencies is unhealthy. * **Clear authority.** Each fact has exactly one home: PostgreSQL owns orders, MongoDB owns products, Elasticsearch owns *no* truth — it is a disposable projection. > ⚠ **A note on what you are reading.** This repository also contains an earlier **Python > FastAPI** implementation (`backend/`, 51 files) that `docker-compose.yml` does **not** build, > and the top-level `README.md` still documents *that* one (Swagger at `:8000/docs`, > `python -m pytest`). The service described throughout this document is the **Java** one — > `backend-java/`. See [`FILE_BY_FILE_GUIDE.md` §0](FILE_BY_FILE_GUIDE.md#0-repository-root--what-is-actually-here). --- ## 2. High-level architecture ### The actual diagram ```*



└──────────────────────────────┤ JAVA WORKER — java-worker service │ │ SAME jar, SAME entrypoint, no profile │ │ com.meridian.backend.messaging │ │ OrderSyncListener (@RabbitListener) │ │ (backend also runs this listener, │ │ so both are competing consumers) │ └──────────────────────────────┘ ``` ### Explaining every arrow | # | Arrow | What actually happens | Code | |---|---|---|---| | — | Browser → Frontend | Vue app served by Vite (dev) or nginx (prod). Bundled SVG artwork is served **by the frontend origin** (`/products/<SKU>.svg`). | `frontend/public/products/`, `frontend/nginx.conf` | | — | Frontend → Backend | axios `fetch`-style JSON over HTTP. `Content-Type: application/json` forces a **CORS preflight** `OPTIONS` first, then the real request with `Authorization: Bearer <jwt>`. | `frontend/src/api/client.js` | | (1) | Backend → PostgreSQL | JDBC/Hikari for users, orders, order_items. JPA repositories, `ddl-auto: validate`. | `core/config/DatabaseConfig.java`, `orders/repository/*` | | (2) | Backend → MongoDB | Spring Data MongoDB + `MongoTemplate` for catalog CRUD, facets, and **reading live prices during checkout**. | `config/MongoConfig.java`, `products/ProductService.java` | | (3) | Backend → RabbitMQ | After an order is committed, publish `{order_id, reason}` to queue `orders_sync` via topic exchange `orders.sync.exchange`. | `messaging/SyncPublisher.java`, `config/RabbitMQConfig.java` | | (4) | Backend → Elasticsearch | Admin search (reads) and direct indexing during seed. | `search/SearchService.java`, `search/repository/OrderSearchRepository.java` | | (5) | Backend → filesystem | Product image upload/delete under `app.uploads.dir` (`/app/uploads`), served back through `/uploads/**`. | `images/ImageService.java`, `config/WebConfig.java` | | (6) | Worker → PostgreSQL | Reads the full order (items + user) with one fetch-join, then stamps `orders.search_indexed_at`. | `messaging/OrderSyncListener.java` | | (7) | Worker ← RabbitMQ | `@RabbitListener` on `orders_sync`, manual ack, retry with backoff, `default-requeue-rejected: false`. | `messaging/OrderSyncListener.java` | | (8) | Worker → Elasticsearch | `indexOrder(doc)` writes one document with `_id = order_id`, then stamps (6). | `search/repository/OrderSearchRepository.java` | **Two arrows to be careful about:** * The **backend also listens** to `orders_sync`. `java-worker` is *not* a separate program — it is the **same `app.jar`, same `ENTRYPOINT`, no Spring profile, only with no published port**. So backend and java-worker are **competing consumers** on one queue (roughly doubling consume throughput). See [§7](#7-docker-architecture). * **Checkout itself talks to Elasticsearch only for reads.** The index is written by the sync path (7)–(8), or directly by the seeder. --- ## 3. Database responsibility ### MongoDB — the product catalog **What lives there:** one document per product in the `products` collection. | Field | Type | Why it exists | |---|---|---| | `_id` | string | Mongo id — this is what `order_items.product_id` stores | | `sku` | string (unique index `uniq_products_sku`) | human-facing stock code, e.g. `WM-001` | | `title`, `description` | string | display copy | | `price` | double | **live** price — re-read on every checkout | | `category` | string (index `ix_products_category`) | browsing facet | | `tags` | `string[]` | free-form facets, used for `$all` filtering and tag counts | | `attributes` | `map<string,any>` | flexible per-category specs (`dpi`, `material`, `switch`…) | | `variants` | `[{sku,color,stock}]` | nested array — colour/stock options that belong to *this* product | | `active` | bool (index `ix_products_active`) | soft-delete flag; storefront shows `active:true` only | | `image_url` | string | pointer to artwork (see [§6](#6-image-architecture)) | | `created_at`, `updated_at` | `OffsetDateTime` | provenance | **Why nested/array fields are useful:** a product and its variants are *always* read together and never queried independently. In SQL you would need `products` + `tags` + `attributes` + `variants` JOINs just to render one card. In MongoDB the whole product is one document — one read, no JOIN, and `attributes` can hold a different shape per category without a schema migration. **It is the catalog authority:** a product's current price, title and stock live *only* here. Nothing else is allowed to redefine them. (Orders keep their own frozen copy — see [§4](#4-order-snapshot-semantics).) ### PostgreSQL — users, orders, order_items ``` users orders order_items ──────────────────────── ──────────────────────── ──────────────────────── id INTEGER PK id INTEGER PK id INTEGER PK name VARCHAR(120) order_number VARCHAR(32) UQ order_id FK → orders email VARCHAR(255) UQ user_id FK → users product_id VARCHAR(64) password_hash VARCHAR(255) order_date TIMESTAMPTZ title VARCHAR(255) role VARCHAR(20) status VARCHAR(20) quantity INTEGER created_at TIMESTAMPTZ total_amount NUMERIC(12,2) unit_price NUMERIC(12,2) created_at / updated_at search_indexed_at (nullable) ``` * **Relationships:** `users 1—n orders 1—n order_items`. Cascade is `ALL` from order → items (deleting an order deletes its lines) but `RESTRICT` from user → orders. * **Transaction boundaries:** `OrderService.createOrder()` is `@Transactional`. Inside one transaction it resolves the cart against MongoDB, inserts the `Order` and every `OrderItem`, and computes the total. Only after the commit does it publish to RabbitMQ (`finally` block). If anything non-business fails, `OrderPersistenceException` is thrown and **no data is written**. * **Order source of truth:** the order detail page, status changes and revenue reporting all read PostgreSQL. Elasticsearch is never authoritative for an order. * **Snapshot semantics:** see [§4](#4-order-snapshot-semantics). ### Elasticsearch — the search projection **What is indexed:** one document per order in index `orders` (1 shard, 0 replicas), built by `ElasticsearchOrderDocument`: ``` order_id long (also the document _id) order_number keyword status keyword order_date date (strict_date_optional_time||epoch_millis) updated_at date total_amount double customer object { id: long, name: search_as_you_type(+keyword), email: search_as_you_type(+keyword) } items NESTED { product_id: keyword, title: search_as_you_type(+keyword), quantity: integer, unit_price: double, line_total: double } ``` **Why it exists:** answering "orders from Wendy in August over ₹200 containing *wireless*, show revenue and status counts" across thousands of rows in milliseconds is exactly what an inverted index + aggregations are for. Doing that in SQL needs a full scan plus a `GROUP BY`. **What it is NOT authoritative for:** everything. The index can be deleted and rebuilt from PostgreSQL at any time — the seeder does exactly that with `recreateIndex()`. If ES and PostgreSQL disagree, PostgreSQL is right. **Search/filter/aggregation responsibility:** all text search, all status/date/price filtering, pagination totals and the KPI aggregations (`sum(total_amount)` and `terms(status)`) are computed in ES. Nothing else in the system performs order search. ### RabbitMQ — the synchronization bus * **What a message represents:** *"order `<id>` changed, reason `<reason>`"* — nothing more. Payload class: `SyncPublisher.OrderSyncMessage { orderId, reason }`, reason is `order_created` or `status_update`. * **When produced:** after a successful commit of `POST /api/orders`, and after `PATCH /api/orders/{id}/status`. Publishing never throws — failure is reported in the response's `sync.error` field rather than failing the user's request. * **Who consumes:** `OrderSyncListener` (running in **both** the backend and java-worker containers), which re-projects the order into Elasticsearch and stamps `orders.search_indexed_at`. * **Topology:** topic exchange `orders.sync.exchange` → durable queue `orders_sync` (routing key `orders.sync`). Manual ack, prefetch 1, concurrency 2, 5 retries with exponential backoff (10s → ×2 → max 120s), and **no requeue on final failure** (a poison message must not loop forever). ### The Java worker * **What it does:** consumes a sync message, loads the order from PostgreSQL with all items and the customer in **one** `JOIN FETCH`, ensures the ES index exists, writes the document, then stamps `search_indexed_at` so the UI can show "Indexed". * **How it updates Elasticsearch:** `OrderSearchRepository.indexOrder()` with `_id = order_id` — an idempotent overwrite, so replaying a message is safe. * **How it differs from the API process:** **structurally, not by code.** Both run the same jar; the only differences are (a) `java-worker` publishes no port, and (b) it is a separate container, so CPU spent indexing is separate from CPU serving HTTP. There is no separate "worker program" in this repository. --- ## 4. Order snapshot semantics ### What an `order_items` row actually stores | Column | Meaning | |---|---| | `product_id` | the **Mongo `_id`** of the product at purchase time (not the SKU) | | `title` | **frozen copy** of the product title | | `unit_price` | **frozen copy** of the price at that moment | | `quantity` | how many were bought | Note what is *not* stored: the SKU, the description, the category, the image, the live price. ### The concrete example in this system The seeder demonstrates the point on purpose (`SeedRunner.applySnapshotDemo()`): | | Value | |---|---| | Live MongoDB product `WM-001` **after** the demo | **Wireless Mouse Pro — ₹60.00** | | Historical PostgreSQL order line recorded **before** the demo | **Wireless Mouse — ₹50.16** | Both are correct, because they answer different questions: * *"What does this product cost today?"* → MongoDB: ₹60.00, title "Wireless Mouse Pro". * *"What did the customer actually pay for this order?"* → PostgreSQL: ₹50.16 for "Wireless Mouse". ### Why the catalog must never rewrite history 1. **Money already moved.** The customer was charged ₹50.16. If the order line silently followed the catalog to ₹60.00, the order total, any invoice and any refund calculation would become wrong — and you could no longer tell what was actually charged. 2. **Orders are an audit record.** "Who bought what, when, for how much" must be answerable years later, long after products have been renamed, repriced or discontinued. 3. **Products die, orders don't.** A product can be deactivated or deleted (soft-delete sets `active=false`). If orders merely referenced live product rows, deleting a product would corrupt every historical order that mentions it. Copying `title` and `unit_price` into the line removes that coupling entirely. 4. **It keeps reads simple.** Rendering an order needs no JOIN back to MongoDB — everything needed for the line item is already in PostgreSQL. 5. **Search agrees with truth.** The ES document copies the same snapshots, so search results show what was paid, not today's price. **The trade-off, stated honestly:** stored prices can drift from the catalog — that is the *feature*. The cost is that

*`order_items` cannot answer "what is this product's price now", so the checkout path re-reads MongoDB to price a *new* order (`OrderService.resolveCart()` uses `productRepository.findByIds(...)`, rejects missing/inactive items and out-of-range prices). --- ## 5. Search architecture ### End-to-end shape ``` AdminSearchView.vue → searchOrders(payload) frontend/src/api/search.js → POST /api/search/orders (Bearer JWT) SearchController @PreAuthorize ADMIN → SearchService.searchOrders(request) buildQuery(request) ──► Elasticsearch query DSL OrderSearchRepository.search(query, from, size, sort) ──► ES convertResponse(...) ──► SearchResponse → { results, total, page, limit, pages, tookMs, aggregations } ``` ### ES document structure (recap) Top level: `order_id`, `order_number`, `status`, `order_date`, `updated_at`, `total_amount`. Nested: `customer {id, name, email}` (an **object**, not nested) and `items {product_id, title, quantity, unit_price, line_total}` as a true **`nested`** field — required so an item's title can match on one line while its `unit_price` matches on another. ### `search_as_you_type` — yes, it is really present `customer.name`, `customer.email` and `items.title` are mapped as `search_as_you_type` (plus a `keyword` subfield). The omni query uses a **whole-phrase** `multi_match` on `customer.name^3` / `customer.email`, which is what makes typing an entire email or name narrow results instead of widening them. ### Query construction — the real methods All in `search/SearchService.java`: | Method | Role | |---|---| | `buildQuery(SearchRequest)` | Top level: a `bool` whose `must` is the omni text clause (if `query` is non-blank), plus filter clauses. **Falls back to `match_all` when there is nothing to filter on.** | | `buildOmniQuery(String)` | Splits the query into terms and builds an **AND** across per-term clauses, then adds a whole-phrase `customer` clause and an uppercased `order_number` prefix clause as `should` alternatives, with `minimumShouldMatch(1)`. | | `termQuery(String)` | Per-term `should`: flat `multi_match` (`order_number^3`, `customer.name^3`), a **nested** `multi_match` over `items.title^2` / `items.product_id`, and an `order_number` prefix clause. | **Why AND across terms:** with OR semantics, `Wendy Wireless` matched *48* of 48 orders (either word hit everything). With term-AND it correctly narrows to **6**. **Filter clauses** (all `filter`, so they do not affect relevance scoring): | Request field | ES clause | |---|---| | `status[]` | `terms: { status: [...] }` | | `date_from` / `date_to` | `range: { order_date: { gte, lt } }` — `date_to` is **+1 day**, so the end date is inclusive | | `min_price` / `max_price` | `range: { total_amount: { gte, lte } }` | > **Implementation note:** range clauses must be built with the typed client builder > (`.field("...").gte(JsonData.of(...))`). Serialising a literal `{"range":{...}}` string makes > ES reject the query with a 400. ### Pagination and sorting `from = (page-1) * limit`, `size = limit`, with `trackTotalHits: true` so `total` is exact rather than capped at 10,000. Sort is `-order_date, -order_id` (newest first, stable ties). `pages` is computed as `ceil(total / limit)`. ### Aggregations / KPIs Run in the **same** ES request, and — importantly — they are scoped to the **filters**, not to the current page: | KPI | Aggregation | |---|---| | Total revenue | `sum` over `total_amount` | | Pending / Processing / Shipped counts | `terms` over `status` (size 10) | `OrderSearchRepository.parseAggregations()` seeds the three statuses to `0` so KPI cards never render blank. ### Verified behaviour (live, 48 seeded orders) | Query | Result | |---|---| | `match_all` | *48* | | status `PENDING`, `SHIPPED` | *16 each* | | `[PENDING, PROCESSING]` | *32* | | `max_price=100` | *15* | | `min_price=200` | *7* | | `min_price=1000` | *0* | | `date_from=2026-09-25` | *9* | | `date_to=2026-08-01` | *4* | | Aug range | *20* | | wireless + SHIPPED + `max_price=200` | *11* | | paging `limit=5` | total 48, pages 10, results 5 | | `wendy` | *6* | | `Wendy Wireless` | *6 (AND, not 48)* | | `wireless` | *48* | | full email `wendy.wireless@example.com` | *6* | | exact `ORD-000250` | *1* | | prefix `ORD-00024` | *5* | | `mechanical keyboard` | *8* | | `john doe` | *6* | | nonsense | *0* | --- ## 6. Image architecture There are **two distinct image paths**, and the distinction is the key to understanding the image bug that was found and fixed. ### Path A — bundled artwork (what the seeded catalog uses) | Aspect | Reality | |---|---| | Where metadata lives | MongoDB `products.image_url` (declared in `ProductDocument` as `@Field("image_url")`) | | Where files live | **The frontend bundle**: `frontend/public/products/<SKU>.svg` | | URL format | `/products/<SKU>.svg`, e.g. `/products/WM-001.svg` — **relative** | | Who serves the files | the **frontend** origin (`http://localhost:5173`), not the backend | | Docker volume | none — these are build-time static assets | | Formats | SVG only (26 dedicated drawings + `default.svg` placeholder) | ### Path B — admin uploads | Aspect | Reality | |---|---| | Upload endpoint | `POST /api/products/{id}/image` (`multipart/form-data`, field `file`, ADMIN only) | | Delete endpoint | `DELETE /api/products/{id}/image` (ADMIN only) | | Where files live | `${app.uploads.dir}/products` → `/app/uploads/products` inside the container | | Docker volume | `uploads_data:/app/uploads` (mounted on **both** `backend` and `java-worker`) | | URL format | `/uploads/products/<uuid>.<ext>` | | How Spring serves it | `WebConfig.addResourceHandlers()`: `/uploads/**` → `file:${app.uploads.dir}/` | | Allowed formats | `image/jpeg`, `image/png`, `image/webp`, `image/svg+xml` (`.jpg .jpeg .png .webp .svg`) | | Max size | 5 MB (`ImageService.MAX_FILE_SIZE` and `spring.servlet.multipart.max-file-size`) | | Filename | `UUID` hex + allowed extension (falls back to `.webp`) | | Validation | `ImageService.validateImageFile()` → `invalid_image_format` / `image_too_large` | > The `uploads_data` volume currently contains **no files** — no image has been uploaded through > the admin UI yet. Only Path A carries data today. ### How the frontend constructs the `<img src>` `frontend/src/utils/image.js` → `resolveImageUrl(value)`: 1. Not a string / empty → `null` → caller renders the **branded placeholder** (monogram + deterministic pattern). 2. Matches an **absolute URL** (`https://…`, `//cdn/…`) or `data:`/`blob:` → returned untouched. 3. Starts with `/uploads/` **and** an API base is configured → prefixed with `API_BASE_URL` (because uploads are served by the *backend*). 4. Anything else (bundled assets like `/products/WM-001.svg`) → returned **as-is**, so it resolves against the app origin. `ProductThumb.vue` then does `resolveImageUrl(product.image_url)`; on `null`, load error or failed load it falls back to the placeholder. It is used by `ProductCard`, `CartDrawer`, `HeroSection`, `CheckoutLineItems`, `CatalogTable` and `CatalogCardList`. ### The defect that was found — and the fix **Root cause (category A + E): the seed never wrote `image_url` at all.** Evidence gathered before the fix: * MongoDB: **0 of 27** product documents had any `image`/`image_url`/`imageUrl` key. * `GET /api/products`: `image_url` key present but `null` for **26 of 26** items — so the API layer and DTO mapping were fine; the *data* was missing. * `SeedRunner.seedProducts()` contained no reference to `image` anywhere. * The artwork *did* exist: `frontend/public/products/` held **26** `<SKU>.svg` files, and a stray out-of-band script `update_all_images.py` at the repo root mapped `SKU → /products/<SKU>.svg` — but it was never part of the seed, so a re-seed wiped it. * Reachability was already correct: `GET http://localhost:5173/products/WM-001.svg` → `200 image/svg+xml`. So: **the file was never the problem, the static mapping was never the problem, and the frontend URL construction was never the problem — the field was simply never populated.** **Fix applied (smallest durable fix, in `SeedRunner.seedProducts()` only):** ```java String sku = (String) p.get("sku"); doc.setImageUrl(SKU_WITHOUT_ARTWORK.contains(sku) ? DEFAULT_ARTWORK_URL // "/products/default.svg" : "/products/" + sku + ".svg"); // "/products/<SKU>.svg" ``` with `SKU_WITHOUT_ARTWORK = Set.of("CHG-001")` (the one SKU with no dedicated drawing). No schema change, no API change, no frontend change, no redesign of the image system. **Verification after the fix:** | Check | Result | |---|---| | Mongo documents with `image_url` | **27 / 27**, none `null` | | `WM-001` | `/products/WM-001.svg` | | `CHG-001` | `/products/default.svg` | | `GET /api/products` non-null `image_url` | **26 / 26** | | URL fetch — WM-001 / MK-001 / ORG-001 | `200 image/svg+xml` (1578B / 3087B / 1640B) | | URL fetch — default.svg | `200 image/svg+xml` (1422B) | *Not verified:* a rendered browser screenshot — no desktop browser is attached to this session, so confirmation is at data, API and URL level rather than visual. --- ## 7. Docker architecture Ten services in `docker-compose.yml`. | Service | Image / build | Purpose | Ports | Depends on | Volumes | Profile | |---|---|---|---|---|---|---| | `postgres` | `postgres:16-alpine` | orders, users, order_items | `5432:5432` | — | `postgres_data` | — | | `mongodb` | `mongo:7` | product catalog | `27017:27017` | — | `mongodb_data` | — | | `elasticsearch` | `docker.elastic.co/…/elasticsearch:8.15.3` | order search index | `9200:9200` | — | `elasticsearch_data` | — | | `rabbitmq` | `rabbitmq:3.13-management-alpine` | order sync queue | `5672:5672`, `15672:15672` | — | `rabbitmq_data` | — | | `backend` | build `./backend-java` | the HTTP API | `8000:8000` | postgres, mongodb, elasticsearch, rabbitmq | `uploads_data:/app/uploads` | — | | `java-worker` | **same build** | RabbitMQ consumer / indexer | *(none published)* | postgres, elasticsearch, rabbitmq | `uploads_data:/app/uploads` | — | | `frontend` | build `./frontend` (`target: dev`) | Vite dev server | `5173:5173` | backend | `./frontend:/app`, `frontend_node_modules` | — | | `frontend-prod` | build `./frontend` (`target: prod`) | nginx static + proxy | `8080:80` | backend | — | `prod` | | `seed` | **same build** | one-shot database seeder | — | postgres, mongodb, elasticsearch | — | `tools` | | `reindex` | **same build** | intended one-shot reindexer | — | postgres, elasticsearch | — | `tools` | Top-level volumes: `postgres_data`, `mongodb_data`, `elasticsearch_data`, `rabbitmq_data`, `frontend_node_modules`, `uploads_data`. ### Key environment variables Shared by `backend` and `java-worker` via the `x-backend-environment` anchor: ``` POSTGRES_HOST=postgres POSTGRES_PORT=5432 MONGO_URI=mongodb://mongodb:27017 SPRING_DATA_MONGODB_URI=mongodb://mongodb:27017 ELASTICSEARCH_URL=http://elasticsearch:9200 RABBITMQ_HOST=rabbitmq RABBITMQ_URL=amqp://<user>:<pass>@rabbitmq:5672/ SERVER_PORT=8000 ``` Frontend: `VITE_API_BASE_URL` (default `http://localhost:8000`) and `VITE_PROXY_TARGET` (`http://backend:8000`). Also relevant: `app.cors.allowed-origins` (default `http://localhost:5173`), `app.uploads.dir` (`/app/uploads`), `app.jwt.secret`, `app.jwt.expiration-minutes` (1440). ### Internal Docker hostnames vs `localhost` This trips people up constantly: | From | To a container | To the host | |---|---|---| | Browser / host shell | use **published ports**: `localhost:8000`, `localhost:5173`, `localhost:9200` | — | | Another container on `app_net` | use the **service name**: `postgres`, `mongodb`, `elasticsearch`, `rabbitmq`, `backend` | `host.docker.internal` (not configured here) | | The backend itself | `mongodb:27017`,*

*`elasticsearch:9200`, `rabbitmq:5672` | — | So: **`localhost` never resolves to a sibling container.** `spring.data.mongodb.uri` inside the backend is `mongodb://mongodb:27017`, not `localhost:27017`. Conversely `VITE_API_BASE_URL=http://localhost:8000` is correct *because it is evaluated in the browser*, where `localhost` is the host machine. ### Run commands ```bash docker compose up -d # infra + backend + java-worker + frontend docker compose build backend seed # required before re-seeding docker compose --profile tools run --rm seed # re-seed (never exits — see caveats) docker compose --profile tools run --rm reindex # ⚠ currently a no-op (see caveats) docker compose --profile prod up -d --build frontend-prod # nginx frontend on :8080 ``` --- ## 8. API map **16 application endpoints** (15 business + health), enumerated by scanning every `@GetMapping` / `@PostMapping` / `@PatchMapping` / `@DeleteMapping` under `backend-java/src/main/java`. Two framework `/actuator/*` endpoints also exist but require auth (see caveats). `AUTH` column: **public** = no token needed; **authenticated** = any valid JWT; **ADMIN** = `ROLE_ADMIN`. | METHOD | PATH | PURPOSE | AUTH | DATABASE / SYSTEM | |---|---|---|---|---| | POST | `/api/auth/login` | Verify email+password, issue JWT | public | PostgreSQL | | POST | `/api/auth/logout` | Acknowledge logout (stateless — nothing revoked) | public | — | | GET | `/api/auth/me` | Current user from the token | public* | PostgreSQL | | GET | `/api/products` | List/filter/sort/paginate catalog | public | MongoDB | | GET | `/api/products/facets` | Category + tag facet counts | public | MongoDB | | GET | `/api/products/{id}` | Single product | public | MongoDB | | POST | `/api/products` | Create product | ADMIN† | MongoDB | | PATCH | `/api/products/{id}` | Partial update | ADMIN† | MongoDB | | DELETE | `/api/products/{id}` | Soft delete (`active=false`) | ADMIN† | MongoDB | | POST | `/api/products/{id}/image` | Upload image (multipart) | ADMIN† | Filesystem + MongoDB | | DELETE | `/api/products/{id}/image` | Remove image | ADMIN† | Filesystem + MongoDB | | POST | `/api/orders` | Checkout — create order | authenticated | PostgreSQL + MongoDB + RabbitMQ | | GET | `/api/orders/{id}` | Order detail (ownership enforced) | authenticated | PostgreSQL | | PATCH | `/api/orders/{id}/status` | Change order status | ADMIN | PostgreSQL + RabbitMQ | | POST | `/api/search/orders` | Admin search + filters + KPIs | ADMIN | Elasticsearch | | GET | `/api/health` | Dependency health + latency | public | all four (read-only) | * `GET /api/auth/me` is `permitAll`, so calling it *without* a token reaches the controller with a `null` principal and returns **500** instead of 401 — see [§10](#10-known-caveats). † ADMIN is enforced by **method-level** `@PreAuthorize("hasRole('ADMIN')")`, not by the URL rules — the URL rules for these paths are shadowed by an earlier `permitAll`. See [§10](#10-known-caveats) for the status-code consequence. ### URL authorization rules as declared (`SecurityConfig.securityFilterChain`) ``` permitAll /api/auth/** , /api/health , /api/users permitAll /api/products , /api/products/facets , /api/products/** , /uploads/** authenticated POST /api/orders , GET /api/orders/** hasRole(ADMIN) PATCH /api/orders/**/status , /api/search/** hasRole(ADMIN) POST /api/products , PATCH /api/products/** , DELETE /api/products/** POST /api/products/**/image , DELETE /api/products/**/image anyRequest -> authenticated ``` Matchers are evaluated **in order and the first match wins** — so the `permitAll` lines for `/api/products` (any method) precede and shadow the ADMIN lines for the same paths. The `@PreAuthorize` annotations on the controller methods are therefore what actually protects product writes. --- ## 9. Important design decisions | Decision | Why | |---|---| | **MongoDB for the catalog** | Products are nested documents (`tags[]`, `attributes{}`, `variants[]`) always read whole. One read, no JOIN, and `attributes` can vary by category without schema changes. | | **PostgreSQL for orders** | Orders must be ACID, referentially intact and permanently auditable. `@Transactional` checkout, FK constraints and `NUMERIC` money types are exactly right for a source of truth. | | **Elasticsearch for search** | Filtering + aggregating across orders is an inverted-index problem. Doing it in SQL would mean scans and `GROUP BY`s on every dashboard load, and ES gives exact `track_total_hits` pagination for free. | | **RabbitMQ for synchronization** | Decouples "order placed" (must be fast and durable) from "search updated" (can lag). Publish-after-commit keeps checkout fast even if the search cluster is slow or down, and the durable queue survives a backend restart. | | **Java worker for the ES projection** | Keeps indexing off the request thread and lets indexing scale independently of HTTP. Because it shares the jar, the projection logic has exactly one implementation — no drift between API and worker. | | **Snapshots for historical order data** | `title` and `unit_price` are copied into `order_items` so catalog changes can never rewrite what a customer paid (see [§4](#4-order-snapshot-semantics)). | | **JWT for authentication** | Stateless sessions: no server-side session store, any number of backend replicas can validate a token, and the browser just stores a string. Stateless also means **logout is client-side only** — the token stays valid until `exp`. | | **BCrypt for passwords** | Adaptive, salted hashing — a stolen `password_hash` cannot be reversed, and cost can be raised later without invalidating existing hashes. Seeded hashes are `$2a$10$…`, 60 chars. | | **Manual Mongo wiring (`MongoConfig`)** | `MongoAutoConfiguration` is excluded so `OffsetDateTime ⇄ BSON Date` conversion is explicit and deterministic. A prior bug here caused a codec failure; the fix is a `MongoCustomConversions` pair, deliberately **not** a custom `CodecProvider`. | | **`search_as_you_type` + `nested` items** | `search_as_you_type` gives as-you-type/phrase matching over customer and item titles; `nested` is required so item fields match per-line rather than across lines. | | **Two-phase order number (`TMP-` → `ORD-%06d`)** | `order_number` is `NOT NULL` and unique, but the sequence value is only known after the first insert, so a provisional `TMP-<uuid>` is written first and replaced with `ORD-%06d` in the same transaction. | | **Soft delete for products** | `active=false` keeps historical orders and search results intact instead of orphaning them. | --- ## 10. Known caveats Taken from the latest [`FINAL_SYSTEM_AUDIT.md`](FINAL_SYSTEM_AUDIT.md) plus findings from this verification pass. Nothing here is hidden or softened. ### Fixed in this pass | Caveat | Status | |---|---| | `OPTIONS` preflight returned **403 `Invalid CORS request`** while simple `GET`s returned 200 | **FIXED** — see [`SYSTEM_WORKFLOW.md` Appendix C](SYSTEM_WORKFLOW.md#appendix-c--admin-login-issue-diagnosis) for the full root cause | | Product images not visible (`image_url` always `null`) | **FIXED** — see [§6](#6-image-architecture) | | Frontend test suite **25 passed / 3 failed** (all CORS-caused) | **NOW 28 passed / 0 failed** (`vitest run`, 5 files) | ### Open — pre-existing, deliberately not changed | # | Caveat | Evidence / impact | |---|---|---| | 1 | **No *Java* backend test suite.** `backend-java/src/test` contains **0 files** (the directory tree exists but is empty) — `mvn test` → "No tests to run". | The 16 endpoints, the auth matrix and the snapshot logic are covered only by manual verification. | 1b | **The repository ships two backends, and the README documents the wrong one.** `backend/` is a **legacy Python FastAPI service** (51 `.py` files) that `docker-compose.yml` never builds — every `build.context` is `./backend-java`. `README.md` still describes FastAPI (`http://localhost:8000/docs` Swagger, `python -m pytest` (101 tests)), and `docs/MERIDIAN_COMPLETE_SYSTEM_KNOWLEDGE.md` documents `backend/app/api/products.py`. Root `tests/` (9 Python files) + `pytest.ini` target that legacy service; `pytest` is not installed in this environment, so those tests were never run here. | A reader following `README.md` would start the wrong service. The running system is the **Java** one — see [`FILE_BY_FILE_GUIDE.md`](FILE_BY_FILE_GUIDE.md) §0. | 2 | **RabbitMQ → worker path has never been exercised by a live checkout end to end.** | The seeder indexes orders directly (`reindexSearch()`), which bypasses the queue. Publishing and listening are individually wired and the topology exists, but no automated test drives checkout → queue → worker → ES. | 3 | **Seed keeps Tomcat alive.** `docker compose --profile tools run --rm seed` never returns; `EXIT=124` is the normal timeout, not a failure. | Success must be judged by the log marker `Seed data initialization completed!`. | 4 | **One future-dated order** — `future_dated_orders = 1` (≈14 h ahead). | Pre-existing in seed data: `ageDays(5)==0` combined with `.withHour((5*13)%24)`. Harmless but visible in date filters. | 5 | **`GET /api/auth/me` without a token → 500.** | `/api/auth/**` is `permitAll`, so `@AuthenticationPrincipal User` is `null` and `AuthService.mapToUserResponse()` NPEs. Log: `Cannot invoke "…User.getId()" because "user" is null`. Should be 401. The frontend never hits this (`fetchMe()` no-ops without a token). | 6 | **Forbidden admin writes → 500 instead of 403/401.** `POST /api/products` as a customer or anonymously returns **500**. | `@PreAuthorize` throws `AccessDeniedException`, but `GlobalExceptionHandler`'s catch-all `@ExceptionHandler(Exception.class)` converts it to `500 internal_error` before Spring Security's `ExceptionTranslationFilter` can map it to 403/401. **The request is still correctly denied** (verified: admin could immediately create the same SKU afterwards, proving no write occurred) — only the status code is wrong. Root cause is the same shadowing noted in [§8](#8-api-map): URL rules match `permitAll` first, so enforcement falls through to method security. | 7 | **Unsupported HTTP method → 500 instead of 405.** e.g. `GET /api/orders` → 500. | `HttpRequestMethodNotSupportedException` is swallowed by the same catch-all. Confirmed: `Unhandled error: Request method 'GET' is not supported`. | 8 | **The `reindex` compose service is a no-op.** | It runs `java -jar app.jar --spring.profiles.active=reindex`, but `@Profile("seed")` on `SeedRunner` is the **only** `@Profile` in the codebase and there is no `application-reindex.yml`. The app starts, does no reindexing, and idles. | 9 | **`java-worker` is not a distinct worker program.** | Same image, same `ENTRYPOINT ["java","-jar","app.jar"]`, no profile, only no published port. Backend and java-worker are **competing consumers** on `orders_sync`. The design docs should not imply a separate worker codebase. | 10 | **Cart is not persisted.** `stores/cart.js` has no `localStorage` write. | Reloading the page empties the cart. | 11 | **No product detail page.** There is no `/products/:id` route. | Product information only appears in cards, hero, cart lines, checkout lines and the admin tables. | 12 | **`order_items.product_id` is the Mongo `_id`, not the SKU.** | Joining a line back to a human-readable SKU requires a Mongo lookup. | 13 | **`app.jwt.algorithm: HS256` is declared but never read.** | `JwtTokenProvider` calls `signWith(secretKey)`, so jjwt infers the algorithm from key length. Observed tokens are actually **`{"alg":"HS384"}`**. | 14 | **Dead code / unused wiring** (harmless, noted for accuracy). | `OrderService.listUsers()` has no mapped `GET*

*/api/users` (though `permitAll` allows the path); `ProductService.ensureIndexes()` is an empty uncalled method; `PaginationMeta` is unreferenced; several repository query methods have no callers; `SyncPublisher` is injected into `SearchService` but unused; `@EnableAsync`/`@EnableScheduling` exist with no `@Async`/`@Scheduled` methods; frontend `api/users.js` is imported by nothing. | | 15 | **No visual browser verification for this pass.** | No desktop browser is attached to this session, so image rendering and login were verified at data, API and URL level, not by a screenshot. | ### System state at the time of writing | Fact | Value | |---|---| | Containers | 7 up, all healthy (`postgres`, `mongodb`, `elasticsearch`, `rabbitmq`, `backend`, `java-worker`, `frontend`) | | Users | 9 (8 `CUSTOMER` + 1 `ADMIN`) | | Products | 27 documents, 26 active, 1 inactive (`CHG-001`) | | Products with `image_url` | **27 / 27** | | Orders | 48 (`ORD-000293` … `ORD-000340`; the range **advances on every seed run**) | | `order_items` | 107, spread over all 48 orders | | Null `order_number` | **0** | | Status split | PENDING 16 / PROCESSING 16 / SHIPPED 16 | | Revenue | ₹2390.96 / ₹1947.52 / ₹2693.96 (total ₹7032.44) | | Elasticsearch `orders` docs | 48 | | Future-dated orders | 1 | | Frontend tests | 28 passed / 0 failed | --- *Next:* [`SYSTEM_WORKFLOW.md`](SYSTEM_WORKFLOW.md) for step-by-step request flows, [`FILE_BY_FILE_GUIDE.md`](FILE_BY_FILE_GUIDE.md) for the source-level tour. ---------- system workflow -> # Meridian — System Workflow > **Part A of the system knowledge package.** > [`SYSTEM_ARCHITECTURE.md`](SYSTEM_ARCHITECTURE.md) — what the system *is* and why it is shaped > that way. [`FILE_BY_FILE_GUIDE.md`](FILE_BY_FILE_GUIDE.md) — what every source file does. This document traces **actual requests through actual classes**. Every class and method named below exists in the source; nothing is hypothetical. Each flow is written as the stack trace of a request: ``` Frontend → endpoint → controller → service → repository/client → database/system → response ``` --- ## Contents | Part | Section | |---|---| | **1** | [§4 — Complete request flow (A…R)](#4-complete-request-flow) | | **2** | [§5 — Authentication & authorization](#5-authentication--authorization) | | **3** | [§14 — Common debugging paths](#14-common-debugging-paths-if-x-is-broken-where-do-i-look) | | **B** | [Appendix B — Image diagnosis](#appendix-b--image-issue-diagnosis) | | **C** | [Appendix C — Admin login diagnosis](#appendix-c--admin-login-issue-diagnosis) | --- ## 4. Complete request flow ### A. Application startup ``` java -jar app.jar (Dockerfile ENTRYPOINT) └─ MeridianApplication.main() @SpringBootApplication(exclude=MongoAutoConfiguration) │ @EnableAsync @EnableScheduling ├─ spring.properties → excludes spring Mongo auto-config entirely ├─ AppConfig reads every env var (Postgres/Mongo/ES/Rabbit/JWT/uploads) ├─ DatabaseConfig HikariDataSource → EntityManagerFactory (ddl-auto: validate) │ → JpaTransactionManager → scans orders/repository ├─ MongoConfig MongoClients.create(uri) [SINGLE MongoClient bean] │ + MongoCustomConversions(OffsetDateTime⇄Date) │ → SimpleMongoClientDatabaseFactory → MongoTemplate ├─ ElasticsearchConfig RestClient → RestClientTransport(+JavaTimeModule mapper) │ → ElasticsearchClient ├─ RabbitMQConfig exchange orders.sync.exchange, queue orders_sync, │ binding, Jackson2JsonMessageConverter, RabbitTemplate(mandatory) ├─ WebConfig @Primary CorsConfigurationSource + CorsFilter │ + ObjectMapper(SNAKE_CASE, JavaTimeModule) │ + /uploads/** → file:/app/uploads/ ├─ SecurityConfig securityFilterChain (stateless, CSRF off, JWT filter), │ BCryptPasswordEncoder, DaoAuthenticationProvider ├─ OrderSyncListener @RabbitListener starts consuming orders_sync └─ SeedRunner @Profile("seed") ONLY — runs on startup when that profile is on seedUsers → seedProducts → seedOrders → applySnapshotDemo → reindexSearch ``` **Ports:** backend binds `:8000`. Readiness is a `wget` spider of `/api/health`. --- ### B. Customer login ``` LoginForm.vue (submit) │ client-side: validateEmail() → /^[^\s@]+@[^\s@]+\.[^\s@]+$/ │ validatePassword() → non-empty ▼ stores/auth.js → login(email, password) │ POST /api/auth/login { "email": "...", "password": "..." } ← snake_case not needed here │ Origin: http://localhost:5173 │ │ ① browser sends OPTIONS preflight (Content-Type: application/json is non-simple) │ → CorsFilter → CorsConfigurationSource → 200 + Access-Control-Allow-* [was 403] │ ② real POST ▼ AuthController.login(@Valid LoginRequest) POST /api/auth/login │ bean validation first: @NotBlank @Email on `email` │ failure → 422 validation_error (GlobalExceptionHandler) ▼ AuthService.authenticate(request) │ userRepository.findByEmail(email) │ ├─ missing → UnprocessableException (422) │ ├─ empty hash → UnprocessableException (422) │ └─ found │ passwordEncoder.matches(raw, $2a$10$...) ← BCrypt │ └─ false → UnprocessableException (422) │ jwtTokenProvider.generateToken(String.valueOf(user.getId())) ▼ LoginResponse { access_token, token_type:"bearer", user{id,name,email,role} } ▼ auth store: localStorage['meridian.auth.token'], ['meridian.auth.user'] → router.push(redirect || '/') ``` --- ### C. Admin login **Identical to [§B](#b-customer-login) ** — the code path has no branch on role. The only differences are the credentials and what happens next: * identity is an **email**: `admin@example.com`, not `admin` (see [Appendix C](#appendix-c--admin-login-issue-diagnosis)) * `user.role == "ADMIN"` is stored in `localStorage` → `authStore.isAdmin` * the router guard then permits `/admin/**` instead of bouncing to the storefront ```js isAdmin: (state) => state.user?.role === 'ADMIN' ``` --- ### D. Storefront product loading ``` StorefrontView.vue (watch on route query q/category/sort/page) │ ├─ listProducts({page, limit, sort, q, category, visibility}) │ frontend/src/api/products.js → GET /api/products?... ▼ ProductController.listProducts(...) GET /api/products [public] │ params: category, tags, q, visibility(active|inactive|all), │ page(≤10000), limit(≤100), sort(newest|title_asc|price_asc|price_desc) ▼ ProductService.listProducts(...) │ Criteria: active flag, category =, tags $all, q → $regex (case-insensitive │ over title|description|sku|tags) │ Sort mapping, mongoTemplate.count() + mongoTemplate.find() │ → ProductListResponse { items, page, limit, total, pages } ▼ MongoDB catalog.products ▲ │ GET /api/products/facets → ProductService.listFacets() │ Aggregation.facet(): category counts + top-40 tag counts (active only) ▼ ProductGrid.vue → ProductCard.vue → ProductThumb.vue (renders image_url or placeholder) ``` --- ### E. Product search (storefront) The storefront "search" is **MongoDB text filtering**, not Elasticsearch: ``` StorefrontLayout.vue (debounced 260 ms, keys / and Ctrl/Cmd+K) → writes ?q= into the URL (URL is the single source of truth) → StorefrontView re-runs D above → ProductService: $regex over title|description|sku|tags ``` **Elasticsearch search is a separate, admin-only feature** — flow [L](#l-admin-search). --- ### F. Add to cart ``` ProductCard.vue / HeroSection.vue "Add to cart" → useCartStore().add(product, qty) ▼ stores/cart.js items[product.id] = { id, sku, title, price, category, image_url, quantity } │ • snapshot of title/price taken NOW (display only) │ • merge if the line exists, clamp quantity to 1..100 (MIN_QTY/MAX_QTY) │ • NOT persisted — reloading empties the cart ▼ CartDrawer.vue (opened by StorefrontLayout cart button) qty −/input/+, remove, subtotal, "Proceed to checkout" ``` > No HTTP request happens here. The cart is purely client-side until checkout. --- ### G. Checkout ``` CheckoutView.vue (on mount) │ Promise.allSettled(cart.lines.map(l => getProduct(l.id))) ← LIVE re-price │ └─ a line that404s or fails is marked "No longer available" and cannot be ordered │ subtotal recomputed from the LIVE prices ▼ "Place order" → createOrder({ items: [{ product_id, quantity }] }) NOTE: no price and no title are sent — the server re-prices everything ``` --- ### H. Order creation → I. PostgreSQL transaction Both steps happen inside one `@Transactional` method: `OrderService.createOrder()`. ``` frontend/src/api/orders.js → POST /api/orders (Authorization: Bearer …) ▼ OrderController.createOrder(@Valid OrderCreate, @CurrentUser User) [authenticated] │ → 201 Created ▼ OrderService.createOrder(payload, user) @Transactional │ │ 1. userId is FORCED from the principal — the payload's userId is ignored │ 2. resolveCart(payload.items) [reads MongoDB] │ • merge duplicate product lines │ • enforce MAX_LINE_QUANTITY = 100 │ • productRepository.findByIds(ids) ← Mongo _id list │ • reject: missing / inactive / price out of 0.01..1_000_000 → 422 │ • MoneyUtils.calculateTotal() (BigDecimal, 2dp HALF_UP) │ │ 3. order = new Order(); order.setOrderNumber("TMP-" + UUID…) ← provisional │ orderRepository.save(order) ← FIRST INSERT │ order.setOrderNumber(String.format("ORD-%06d", order.getId())) ← final │ orderRepository.save(order) ← UPDATE same row │ (two-phase because order_number is NOT NULL + UNIQUE and the id is only │ known after insert; both saves are in ONE transaction) │ │ 4. for each resolved line: new OrderItem(productId, title, unitPrice, qty) │ order.getItems().add(item) → cascade ALL persists the lines │ order.recalculateTotal() │ │ 5. any non-AppException → OrderPersistenceException("No data was written") → ROLLBACK │ │ └─ finally: syncPublisher.publishOrderSync(id, "order_created") [after commit] ▼ OrderCreateResponse { … order…, sync: { enqueued, queue, error } } ``` **PostgreSQL transaction boundary (flow I):** ``` BEGIN INSERT orders (order_number 'TMP-…', user_id, order_date, status, total) UPDATE orders (order_number 'ORD-%06d') INSERT order_items × N (product_id, title snapshot, quantity, unit_price) UPDATE orders (total_amount, updated_at) COMMIT ← only now is the order durable -- then, outside the transaction: publish → RabbitMQ ``` If the broker is unreachable, **the order still exists** — `SyncInfo.enqueued=false` with the error is returned in the response body rather than failing checkout. --- ### J. RabbitMQ synchronization ``` SyncPublisher.publishOrderSync(orderId, reason) messaging/SyncPublisher.java │ RabbitTemplate.convertAndSend("orders_sync", OrderSyncMessage{orderId, reason}) │ RabbitTemplate is built with mandatory=true + a Jackson2JsonMessageConverter │ failures are caught and returned as SyncInfo{enqueued:false, error} ▼ RabbitMQConfig topology: exchange orders.sync.exchange (topic, durable)*

queue orders_sync (durable) binding orders.sync.exchange → orders_sync, routing key "orders.sync" ``` `OrderSuccessPanel.vue` surfaces the `sync.enqueued` flag to the user. --- ### K. Java worker → Elasticsearch ``` RabbitMQ orders_sync ▼ OrderSyncListener.handleOrderSync(OrderSyncMessage) @RabbitListener │ manual ack · prefetch 1 · concurrency 2 │ retry: 5 attempts, 10s initial, ×2, max 120s │ default-requeue-rejected = false ← a poison message is NOT requeued forever ▼ syncOrderDocument(orderId, reason) (private) │ 1. orderRepository.findByIdWithItemsAndUser(orderId) │ JOIN FETCH items + user ← ONE query, no N+1 │ 2. orderSearchRepository.ensureIndex() ← create orders index if absent │ 3. doc = buildOrderDocument(order) ← same mapping as SeedRunner │ 4. orderSearchRepository.indexOrder(doc) │ _id = order_id, Refresh.True when app.elasticsearch.refresh-on-write=true │ 5. orderRepository.updateSearchIndexedAt(orderId, now) ← stamps Postgres ▼ Elasticsearch index `orders` ▲ └─ OrderDetailsView → SearchSyncIndicator shows "Indexed" / "Not yet indexed" ``` > **Important:** `java-worker` and `backend` both run this listener — competing consumers. > Additionally, the **seeder bypasses this path entirely** (`SeedRunner.reindexSearch()` does > `recreateIndex()` + `bulkIndexOrders()` directly). --- ### L. Admin search ``` AdminSearchView.vue (350 ms debounced; any filter change resets page to 1) ▼ frontend/src/api/search.js → POST /api/search/orders { query, status[], date_from, date_to, min_price, max_price, page, limit } ▼ SearchController.searchOrders(@Valid SearchRequest, @CurrentUser User) @PreAuthorize("hasRole('ADMIN')") + URL rule /api/search/** = ADMIN ▼ SearchService.searchOrders(request) │ buildQuery(request) │ ├─ text? buildOmniQuery(text): term-AND across terms │ │ + phrase multi_match on customer.name^3 / customer.email │ │ + order_number prefix (uppercased) │ │ minimumShouldMatch(1) │ │ └─ per term: termQuery() → flat multi_match (order_number^3, │ │ customer.name^3) | NESTED multi_match (items.title^2, │ │ items.product_id) | order_number prefix │ ├─ status[] → terms filter │ ├─ date_from/to → range order_date (to is +1 day, inclusive) │ ├─ min/max_price → range total_amount │ └─ nothing at all → match_all │ from = (page-1)*limit, size = limit, trackTotalHits: true │ sort = -order_date, -order_id ▼ OrderSearchRepository.search(query, from, size, sort) │ + aggregations: sum(total_amount) → revenue │ terms(status, 10) → status_counts (PENDING/PROCESSING/SHIPPED seeded 0) ▼ Elasticsearch index orders ▲ SearchResponse { results[], total, page, limit, pages, took_ms, aggregations } ▼ SearchKpis (revenue + counts) · SearchFilters · ResultsTable / ResultsCardList · AppPagination ``` --- ### M. Admin order details ``` Route /admin/orders/:id → OrderDetailsView.vue ▼ frontend/src/api/orders.js → GET /api/orders/{id} Authorization: Bearer … ▼ OrderController.getOrder(@PathVariable Integer, @CurrentUser User) │ • OrderService.getOrder() → 404 order_not_found │ • non-ADMIN asking for someone else's order → 403 ▼ OrderService.getOrder(id) │ orderRepository.findByIdWithItemsAndUser(id) ← Postgres, JOIN FETCH ▼ OrderResponse { id, order_number, customer{id,name,email,role}, order_date, status, total_amount, updated_at, search_indexed_at, items[] } ▼ "Source of truth: PostgreSQL" badge · OrderStatusControl · SearchSyncIndicator Customer / Totals / Line-items cards ``` --- ### N. Customer order details **The same endpoint and the same code path as [M](#m-admin-order-details)** — there is no separate customer order API. The difference is purely authorization: a `CUSTOMER` may read only their own order (`OrderController.getOrder` compares `order.getUser().getId()` with the principal and returns **403** on a mismatch; a missing id returns **404**). Verified: `john.doe` reading an order owned by `riley.brooks` → `403`; admin → `200`; no token → `401`. > Note the frontend only *routes* to `/admin/orders/:id`, so order details are reachable from > the admin area. --- ### O. Admin status update ``` OrderStatusControl.vue (BaseSelect: PENDING / PROCESSING / SHIPPED) ▼ frontend/src/api/orders.js → PATCH /api/orders/{id}/status { status } ▼ OrderController.updateStatus(...) @PreAuthorize("hasRole('ADMIN')") + URL rule PATCH /api/orders/**/status = ADMIN ▼ OrderService.updateOrderStatus(id, status) @Transactional │ load → 404 if missing │ order.setStatus(status); order.setUpdatedAt(now) │ commit │ finally: syncPublisher.publishOrderSync(id, "status_update") │ • unchanged status → SyncInfo{queue:"in-process"} (no message published) ▼ StatusUpdateResponse { …order…, sync } → OrderDetailsView updates the badge + toast → RabbitMQ → worker → Elasticsearch re-index (flow K) ``` --- ### P. Catalog administration ``` CatalogAdminView.vue │ list: listProducts({page,limit,sort,q,category,visibility}) │ facets: getProductFacets() ▼ READ GET /api/products → ProductService.listProducts [public] GET /api/products/facets → ProductService.listFacets [public] CREATE POST /api/products → @PreAuthorize ADMIN → createProduct │ duplicate sku → ConflictException 409 │ price ≤ 0 or > 1_000_000 → 422 │ normalizeTags() → productRepository.save() UPDATE PATCH /api/products/{id} → @PreAuthorize ADMIN → updateProduct │ partial Update via mongoTemplate.updateFirst │ sku uniqueness via findBySkuAndIdNot → 409 │ stamps updated_at DELETE DELETE /api/products/{id} → @PreAuthorize ADMIN → deleteProduct SOFT delete: active = false (history and search results stay intact) ▼ MongoDB catalog.products ``` `ProductFormModal.vue` validates first (`utils/validation.js`) and maps a 422 back onto fields via `fieldErrorsFrom(error)`. --- ### Q. Product image upload ``` ProductFormModal.vue → uploadProductImage(id, file) frontend/src/api/products.js POST /api/products/{id}/image Content-Type: multipart/form-data, field "file" ▼ ImageController.uploadProductImage(...) @PreAuthorize ADMIN ▼ ImageService.uploadImage(productId, file) │ 1. productRepository.findById → 404 product_not_found │ 2. validateImageFile(): type ∈ {image/jpeg, image/png, image/webp, image/svg+xml} │ ext ∈ {.jpg .jpeg .png .webp .svg} │ size ≤ 5 MB → 422 invalid_image_format / image_too_large │ 3. filename = UUID hex + allowed extension (fallback .webp) │ 4. Files.createDirectories(/app/uploads/products); Files.copy(...) ← Filesystem │ 5. if a previous /uploads/... file exists, delete it │ 6. product.setImageUrl("/uploads/products/<uuid><ext>"); save() ← MongoDB ▼ ProductResponse (refreshed) → modal preview ``` --- ### R. Product image display ``` ProductThumb.vue imageUrl = resolveImageUrl(product.image_url) utils/image.js │ ├─ null / empty ──────────────────────────► placeholder (monogram + pattern) ├─ https://… │ //cdn/… │ data: │ blob: ─► returned untouched ├─ starts with "/uploads/" ──────────────────► API_BASE_URL + path │ (uploads are served by the BACKEND) └─ anything else, e.g. "/products/WM-001.svg" ► returned as-is (bundled ARTWORK, served by the FRONTEND) @error → failed = true → falls back to placeholder ``` **Who serves which file:** | `image_url` value | Served by | Verified response | |---|---|---| | `/products/WM-001.svg` | frontend `:5173` (Vite/nginx static) | `200 image/svg+xml` 1578 B | | `/uploads/products/<uuid>.png` | backend `:8000` via `WebConfig` resource handler `/uploads/**` → `file:/app/uploads/` | *(volume is currently empty)* | See [Appendix B](#appendix-b--image-issue-diagnosis) for the diagnosis of the image bug. --- ## 5. Authentication & authorization ### Login endpoint `POST /api/auth/login` with `{ "email": "<email>", "password": "<pw>" }`. There is **no username field** — `LoginRequest` declares `@NotBlank @Email private String email`, and the `users` table has **no `username` column** (columns: `id, name, email, password_hash, role, created_at`). ### Password hashing — BCrypt * `SecurityConfig.passwordEncoder()` → `BCryptPasswordEncoder`. * Seeded hashes are `$2a$10$…`, **60 characters** — a genuine BCrypt hash. * `AuthService.authenticate()` calls `passwordEncoder.matches(raw, stored)`. * Seed writes hashes via `passwordEncoder.encode(...)` (it once wrote plaintext — fixed; see the audit). ### JWT creation ```java // JwtTokenProvider.generateToken(String userId) Jwts.builder().subject(userId).issuedAt(now).expiration(now + 1440min) .signWith(secretKey) // secretKey = Keys.hmacShaKeyFor(app.jwt.secret) .compact(); ``` Observed token (decoded): ```json header {"alg":"HS384"} payload {"sub":"477","iat":1791175280,"exp":1791261680} // 1440 minutes ``` **JWT contents:** `sub` = the **user id as a string**, `iat`, `exp`. Nothing else. > Two things worth knowing: > * **There is no role claim and no email claim.** The role is fetched from PostgreSQL on every > request (below). > * `app.jwt.algorithm: HS256` is declared in `application.yml` but **never read** — the code > calls `signWith(secretKey)`, and jjwt picks the algorithm from the key length, so real > tokens are `HS384`. ### JWT validation & the filter ``` JwtAuthenticationFilter (extends OncePerRequestFilter, registered BEFORE UsernamePasswordAuthenticationFilter) 1. read header Authorization: "Bearer <token>" (absent → continue unauthenticated) 2. jwtTokenProvider.validateToken(token) (parseSignedClaims; any exception → false) 3. userId = jwtTokenProvider.getUserIdFromToken() (claims.getSubject()) 4. userRepository.findById(userId) ← PostgreSQL on EVERY request • token valid but user gone → not authenticated 5. authority = "ROLE_" + user.getRole() ← THIS is what makes hasRole() work 6. SecurityContextHolder.setRequestAuthentication(…) 7. chain.doFilter(...) ``` Step 5 is the crux: `SimpleGrantedAuthority("ROLE_" + "ADMIN")` → `ROLE_ADMIN` → `hasRole("ADMIN")` matches. Without the `ROLE_` prefix, every `hasRole("ADMIN")` would fail. ### Authorities and the two roles | Role | Authority granted | Can do | |---|---|---| | `CUSTOMER` | `ROLE_CUSTOMER` | browse, checkout, read **own** orders | | `ADMIN` | `ROLE_ADMIN` | everything above + search, status changes, catalog writes, image upload, read any order | ### Protected routes Declared in `SecurityConfig.securityFilterChain` — see [`SYSTEM_ARCHITECTURE.md` §8](SYSTEM_ARCHITECTURE.md#8-api-map) for the exact list and the shadowing note. Enforcement actually happens in **two places**: 1. **URL rules** (`AuthorizationFilter`) — effective for `/api/search/**`, `PATCH /api/orders/**/status`, and `POST|GET /api/orders`. 2. **Method-level `@PreAuthorize("hasRole('ADMIN')")`** — effective for product writes and image upload/delete,

*because the URL `permitAll` lines for `/api/products` match first. ### Backend authorization checks * **Admin-only operations:** `@PreAuthorize` on `SearchController.searchOrders`, `OrderController.updateStatus`, `ProductController.create/update/deleteProduct`, `ImageController.upload/removeProductImage`. * **Customer ownership:** `OrderController.getOrder` — non-admin requesting another user's order → **403**; missing id → **404**. * **Server-side identity override:** `OrderService.createOrder` forces `userId` from the principal, so a client cannot place an order as someone else. * **401 / 403 bodies:** `CustomAuthenticationEntryPoint` → `{"error":{"code":"unauthorized", "message":"Could not validate credentials"}}`; `CustomAccessDeniedHandler` → `{"error":{"code":"forbidden","message":"Admin access required"}}`. ### How `admin/admin` is *supposed* to work Based strictly on the current implementation: | You type | What happens | |---|---| | `admin` / `admin` | **422** `validation_error` — `email must be a well-formed email address`. Never reaches password checking. | | `admin@example.com` / `admin` | **200** + JWT with `role: "ADMIN"` ✅ | | `admin@example.com` / wrong password | 422 (invalid credentials) | So "`admin/admin`" is a shorthand for "`admin@example.com` / `admin`". The login form itself says so — `LoginForm.vue` renders the hint: ``` Demo accounts: john.doe@example.com / john, admin@example.com / admin ``` and the field is labelled **Email** with placeholder `you@example.com`. ### Seeded customer accounts From `SeedRunner.seedUsers()` — 9 users in PostgreSQL, all BCrypt-hashed: | id | name | email | password | role | |---|---|---|---|---| | +0 | John Doe | `john.doe@example.com` | `john` | CUSTOMER | | +1 | Jane Smith | `jane.smith@example.com` | `jane` | CUSTOMER | | +2 | Wendy Wireless | `wendy.wireless@example.com` | `wendy` | CUSTOMER | | +3 | Alex Rivera | `alex.rivera@example.com` | `alex` | CUSTOMER | | +4 | Sam Patel | `sam.patel@example.com` | `sam` | CUSTOMER | | +5 | Casey Nguyen | `casey.nguyen@example.com` | `casey` | CUSTOMER | | +6 | Morgan Lee | `morgan.lee@example.com` | `morgan` | CUSTOMER | | +7 | Riley Brooks | `riley.brooks@example.com` | `riley` | CUSTOMER | | +8 | **Admin** | **`admin@example.com`** | **`admin`** | **ADMIN** | (Customer ids start at 460 in the first seed run and advance on every subsequent run.) ### Logout behavior `POST /api/auth/logout` returns a fixed `{"message":"Logged out successfully"}`. Because the session is **stateless JWT**, **nothing is revoked server-side** — the token remains valid until `exp` (24 h). The frontend discards `localStorage['meridian.auth.token'/'meridian.auth.user']`, which is what actually ends the session *in that browser*. Anyone holding a copy of the token can still use it. --- ## 14. Common debugging paths (if X is broken, where do I look?) ### Products don't appear ``` MongoDB catalog.products → countDocuments({active:true}) → ProductRepository (products/repository/) → ProductService.listProducts() ← Criteria, visibility/sort/paging → ProductController.listProducts() GET /api/products → curl http://localhost:8000/api/products?limit=5 (is the API returning them?) → frontend api/products.js listProducts() → stores (none) → StorefrontView.vue watch on route query → ProductGrid.vue → ProductCard.vue ``` Quick checks: is `active` false? Is `visibility` filtering them out? Did the store fail to load (`ProductGrid` shows an `AppAlert` on error)? Note the list returns **26 of 27** — `CHG-001` is seeded inactive. ### Login doesn't work ``` 1. POST /api/auth/login directly with curl ← START HERE, removes the browser from the picture correct credentials → 200 + JWT? 2. PostgreSQL users table → does the email exist? left(password_hash,7) = '$2a$10$'? 3. password → AuthService.authenticate() → passwordEncoder.matches() 4. wrong identifier shape → LoginRequest @Email → 422 "must be a well-formed email address" 5. JWT generated? → JwtTokenProvider.generateToken() (decode header/payload) 6. JwtAuthenticationFilter → extracts Bearer, loads user, grants ROLE_<role> 7. CORS → OPTIONS preflight must be 200 ← this was the real failure; see Appendix C 8. frontend api/client.js → baseURL from VITE_API_BASE_URL, Authorization interceptor 9. stores/auth.js login() → stores token + user in localStorage 10. router/index.js beforeEach → requiresAuth / requiresAdmin guards 11. console/network tab → is the preflight failing? is a 422 being shown as a generic error? ``` ### Images don't appear ``` MongoDB products.image_url → is it null? ← THIS was the bug; see Appendix B → ProductDocument @Field("image_url") → GET /api/products → is image_url non-null in the JSON? → frontend utils/image.js resolveImageUrl() → ProductThumb.vue: imageUrl / @error / placeholder → <img src> — fetch that exact URL by hand /products/<SKU>.svg → must return 200 image/svg+xml from :5173 /uploads/products/… → must return 200 from :8000 (WebConfig /uploads/** handler) → static resource mapping: WebConfig.addResourceHandlers() → filesystem: docker compose exec backend ls -la /app/uploads/products → Docker volume: `uploads_data:/app/uploads` on backend AND java-worker ``` Gotcha: a missing `/products/<SKU>.svg` gets the SPA fallback (`200 text/html`), so `<img>` fails and silently shows the placeholder. ### Search doesn't work ``` Elasticsearch: curl localhost:9200/orders/_count (expect 48) → index exists? OrderSearchRepository.ensureIndex() → mapping: nested items? search_as_you_type customer/title? → SearchService.buildQuery() / buildOmniQuery() / termQuery() • AND across terms (OR makes "Wendy Wireless" match everything) • range clauses need the typed builder, not a literal {"range":…} string • date_to must be +1 day for inclusivity → OrderSearchRepository.search() (aggregations, trackTotalHits) → POST /api/search/orders as ADMIN (401/403 = auth, 400 = bad query, 503 = ES down) → frontend api/search.js → AdminSearchView.vue (350 ms debounce) ``` ### Order doesn't appear in Elasticsearch ``` PostgreSQL orders → does the row exist? search_indexed_at set? → RabbitMQ → queue orders_sync non-empty? (mgmt UI :15672) publish failed? SyncPublisher returns {enqueued:false, error} in the response → OrderSyncListener → is the consumer connected? (backend AND java-worker both consume) manual ack + 5 retries, no requeue on final failure → a poison message is DROPPED → OrderSearchRepository.indexOrder() → _id = order_id (idempotent overwrite) → freshness: app.elasticsearch.refresh-on-write=true → order ids advance on every seed → stale docs cause a transient mismatch; the seeder guards this with recreateIndex() first ``` ### Login works but a page is blank / redirects ``` browser localStorage → meridian.auth.token / meridian.auth.user present? → stores/auth.js fetchMe() (clears the session if it fails) → router beforeEach: requiresAuth && !isAuthenticated → /login?redirect=… requiresAdmin && !isAdmin → / → isAdmin derives from user.role (NOT from decoding the JWT) → network tab: which request 401'd/403'd/500'd? ``` ### Nothing works at all ``` docker compose ps → all 7 containers healthy? curl localhost:8000/api/health → which dependency is "down"? (503 = degraded) docker compose logs backend → look for ERROR + "Unhandled error:" mvn -q compile → build actually succeeded? ``` --- ## Appendix B — Image issue diagnosis **Reported:** *"Images for each product are not visible."* Diagnosed independently of the earlier seed problems; no code was changed until the root cause was proven. ### STEP 1 — Actual MongoDB product documents Inspected `WM-001` and enumerated every top-level key across the collection: ``` {_id, sku, title, description, price, category, tags[], attributes{}, variants[], active, created_at, updated_at, _class} ``` Existence counts for image-ish fields: ``` image: 0 image_url: 0 imageUrl: 0 images: 0 thumbnail: 0 photo: 0 picture: 0 ``` → **MongoDB has no image field at all**, even though `ProductDocument` declares `@Field("image_url") private String imageUrl;`. ### STEP 2 — What `GET /api/products` actually returns ```json { "sku": "ORG-001", "title": "Desk Organizer Bamboo", …, "image_url": null, … } ``` `image_url` key **present** but `null` — **26 of 26 items null**. → The DTO mapping and Jackson serialisation are fine. The *data* is missing. ### STEP 3 — How the frontend builds the `<img src>` `ProductThumb.vue`: `resolveImageUrl(props.product?.image_url)`. `utils/image.js`: empty/null → `null` → `showImage = false` → **branded placeholder**. → Every card rendered the placeholder. That is exactly what "images not visible" looks like. ### STEP 4 — Static resource configuration * `WebConfig.addResourceHandlers()` maps `/uploads/**` → `file:${app.uploads.dir}/` — present and correct. * `app.uploads.dir` = `/app/uploads`; `uploads_data` volume is mounted; the directory exists but is **empty**. * `/uploads/**` → `500` for a non-existent file (the catch-all handler; no file has ever been uploaded). ### STEP 5 — Are representative image URLs reachable? ``` http://localhost:5173/products/WM-001.svg → 200 image/svg+xml 1578B ✅ http://localhost:5173/products/default.svg → 200 image/svg+xml 1422B ✅ http://localhost:5173/products/NOPE-999.svg → 200 text/html (SPA fallback!) ``` → The files exist and are served correctly. ### STEP 6 — Do image files exist in the backend container? `docker compose exec backend ls -la /app/uploads/products` → **empty**. Correct for this design: the seeded artwork lives in the *frontend* bundle (`frontend/public/products/`, **26** `<SKU>.svg` files), not in the backend. SKU ↔ artwork coverage: **26 of 27** SKUs have a drawing; only `CHG-001` does not. ### STEP 7 — ROOT CAUSE **Category A + E — the seed never populated `image_url`.** Confirmed by `grep -n "image" SeedRunner.java` → **no match at all**. A stray out-of-band script `update_all_images.py` at the repo root contained the intended `SKU → /products/<SKU>.svg` mapping, but it was never wired into the seed, so any re-seed wiped it. Ruled out by evidence: API layer ✅ (returns the field), frontend URL construction ✅ (`resolveImageUrl` handles this exact shape), static resource mapping ✅ (`/uploads/**` configured), Docker volume ✅ (mounted), file existence ✅ (26 SVGs, all `200`), port/origin ✅ (relative URL against the app origin). ### Smallest fix applied Only `SeedRunner.seedProducts()`: ```java String sku = (String) p.get("sku"); doc.setImageUrl(SKU_WITHOUT_ARTWORK.contains(sku) ? DEFAULT_ARTWORK_URL : "/products/" + sku + ".svg"); ``` reusing the convention the project already established. No schema/API/frontend/architecture change. ### Verification | Check | Before | After | |---|---|---| | Mongo docs with `image_url` | **0 / 27** | **27 / 27** | | `image_url` null | all | **0** | | `GET /api/products` non-null | **0 / 26** | **26 / 26** | | `WM-001` | — | `/products/WM-001.svg` | | `CHG-001` | — | `/products/default.svg` | | URL fetches | already 200 | **200 `image/svg+xml`** (1578 / 3087 / 1640 / 1422 B) | Seed log: `Products seeded: 27` … `Seed data initialization completed!` **Not verified:** a*

*rendered screenshot — no desktop browser is attached to this session. --- ## Appendix C — Admin login issue diagnosis **Reported:** *"I cannot log into the admin portal."* Traced as `admin`/`admin` through the whole system, without assuming it related to the old seed failure. ### STEP 1 — PostgreSQL user **The table has no `username` column.** `\d users` → `id, name, email, password_hash, role, created_at` (unique index on `email`, check constraint limiting `role` to `CUSTOMER|ADMIN`). ``` id=468 name=Admin email=admin@example.com role=ADMIN hash=$2a$10$… len=60 ``` ### STEP 2 — Is the hash BCrypt? Yes — prefix `$2a$10$`, length **60**. ### STEP 3 — Direct call to the login endpoint (no browser) | Body | HTTP | Response | |---|---|---| | `{"email":"admin@example.com","password":"admin"}` | **200** | JWT issued, `user.role = "ADMIN"` ✅ | | `{"email":"admin","password":"admin"}` | **422** | `email must be a well-formed email address` | | `{"username":"admin","password":"admin"}` | **422** | `email must not be blank` | → **The backend login API works perfectly.** `admin` is simply not a valid identity; the field is `email`. ### STEP 4 — Does the login POST trigger a browser preflight? **Yes.** `Content-Type: application/json` is not a CORS-safelisted content type, so the browser must send an `OPTIONS` preflight first. ``` OPTIONS /api/auth/login Origin: http://localhost:5173 Access-Control-Request-Method: POST Access-Control-Request-Headers: content-type → HTTP/1.1 403 body: "Invalid CORS request" ``` **Before the fix**, every preflight returned 403 regardless of headers, while simple `GET`s returned 200. **The browser therefore never sent the actual POST** — login failed even with correct credentials. Note the subtlety: the *real* POST (when forced with curl) returned **200 with `Access-Control-Allow-Origin: http://localhost:5173`** — so the request handler itself was CORS-correct; only the preflight failed. ### STEP 5 — The actual frontend login request | Aspect | Value | |---|---| | URL | `VITE_API_BASE_URL` → `http://localhost:8000` + `/api/auth/login` | | Method | `POST` | | Headers | `Content-Type: application/json` → **triggers preflight**; `Authorization` added only when a token already exists | | Body | `{ email, password }` — the field is **Email**, placeholder `you@example.com` | | Credentials | not used (`withCredentials` unset) | | Base URL | baked-in absolute `http://localhost:8000` → cross-origin from `:5173` | ### STEP 6 — `JwtAuthenticationFilter` Extracts `Authorization: Bearer …` → `validateToken` → `getUserIdFromToken` → `userRepository.findById` → grants **`ROLE_` + role** → sets the `SecurityContextHolder`. Verified working: the token minted in STEP 3 successfully authenticated `GET /api/auth/me` (200) and `POST /api/search/orders` (200). ### STEP 7 — Admin route protection in Vue `/admin` carries `meta.requiresAuth + requiresAdmin`. `beforeEach`: `requiresAuth && !isAuthenticated` → `/login?redirect=…`; `requiresAdmin && !isAdmin` → `/storefront`. `isAdmin` reads `user.role` from `localStorage`. This layer was never reached, because the request never left the browser. ### STEP 8 — Does the backend accept the JWT? Yes. ``` GET /api/auth/me + admin JWT → 200 {"id":477,…,"role":"ADMIN"} POST /api/search/orders + admin JWT → 200 results GET /api/orders/… no token → 401 POST /api/search/orders no token → 401 POST /api/search/orders customer JWT → 403 GET /api/orders/<other's> customer JWT → 403 ``` ### STEP 9 — The exact failure layer **The CORS preflight.** Not the database, not BCrypt, not the login API, not JWT generation, not the filter, not authorities, not the router. #### Why the preflight failed — root cause proven by bytecode `SecurityConfig` declares `private final CorsConfigurationSource corsConfigurationSource;` — but `WebConfig` only exposed a `@Bean CorsFilter`, and **`CorsFilter` does not implement `CorsConfigurationSource`** (verified by `javap`). Spring Boot 3.2.5 auto-configuration defines **no** `CorsConfigurationSource` bean either (verified by inspecting the jar). So Spring resolved the dependency **by type** to the only remaining candidate: Spring MVC's `HandlerMappingIntrospector`, which *does* implement `CorsConfigurationSource` (bean `mvcHandlerMappingIntrospector`). It returns **`null`** here, because no handler declares its own CORS rules (`aconst_null; areturn` in `getCorsConfiguration`). `DefaultCorsProcessor.processRequest` (Spring 6.1.6) with a `null` configuration — disassembled: ``` 114: aload_1 // config 115: ifnonnull 139 // if (config != null) → handle properly 118: iload 5 // isPreflight 120: ifeq 137 123: rejectRequest() // ← null config + PREFLIGHT → 403 135: iconst_0; ireturn 137: iconst_1; ireturn // ← null config + SIMPLE req → allow ``` | Request type | `config` | Result | |---|---|---| | Simple `GET` with `Origin` | `null` | **allowed** → passes through (and picks up ACAO from the separate container `CorsFilter` bean) — *which is why simple GETs looked healthy* | | **Preflight `OPTIONS`** | `null` | **`rejectRequest()` → 403 `Invalid CORS request`** | That asymmetry is precisely the reported symptom, and it blocked every browser `POST` carrying `Content-Type: application/json` — login included. #### Smallest fix applied `WebConfig` — expose the configuration that was already written as a proper bean: ```java @Bean @Primary public CorsConfigurationSource corsConfigurationSource() { /* /** → allowed origins */ } @Bean public CorsFilter corsFilter(CorsConfigurationSource corsConfigurationSource) { return new CorsFilter(corsConfigurationSource); } ``` `@Primary` makes it win over `HandlerMappingIntrospector`; `corsFilter` now reuses the same source so the two cannot drift. **`SecurityConfig` was not modified.** #### Verification | Check | Before | After | |---|---|---| | `OPTIONS /api/auth/login` preflight | **403** | **200** + `Access-Control-Allow-Origin: http://localhost:5173`, `Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS`, `Allow-Headers: content-type`, `Allow-Credentials: true`, `Max-Age: 3600` | | `OPTIONS /api/search/orders` preflight | 403 | **200** | | Simple `GET /api/health` with `Origin` | 200 | **200** + ACAO + **body intact (full JSON payload)** — no regression | | Preflight from `http://evil.test` | 403 | **403** (origin allow-list still enforced) | | Login `admin@example.com`/`admin` | 200 (curl only) | **200**, `role: ADMIN` | | Admin JWT → `/api/auth/me` | — | **200** | | Admin JWT → `/api/search/orders` | — | **200** | | No token → `/api/orders/…` | — | **401** | | Customer JWT → `/api/search/orders` | — | **403** | | Customer JWT → another user's order | — | **403** | | Frontend test suite | **25 passed / 3 failed** | **28 passed / 0 failed** | #### Conclusion Two independent causes, both real: 1. **Blocking root cause:** the CORS preflight returned 403, so the browser never issued the login request. *(Fixed.)* 2. **Credential form:** `admin`/`admin` is not a valid pair — the identity is an **email**. The correct pair is **`admin@example.com` / `admin`**, which is exactly what the login form's own hint displays. *(Not a code defect.)* --- *See also:* [`SYSTEM_ARCHITECTURE.md`](SYSTEM_ARCHITECTURE.md) for the architecture and caveats, [`FILE_BY_FILE_GUIDE.md`](FILE_BY_FILE_GUIDE.md) for the source-level tour. ---------- this is the file by file guide -> # Meridian — File-by-File Guide > **Part A of the system knowledge package.** > [`SYSTEM_ARCHITECTURE.md`](SYSTEM_ARCHITECTURE.md) — what the system is and why. > [`SYSTEM_WORKFLOW.md`](SYSTEM_WORKFLOW.md) — how a request travels end to end. This is the source-level tour: every meaningful file, what it is for, who calls it, what it calls, and which system it touches. **76 Java files** under `backend-java/src/main/java/`, **70 files** under `frontend/src/`. Each entry follows a fixed shape: ``` Purpose — what the file does Used by — who calls it Calls — what it calls Database — PostgreSQL / MongoDB / Elasticsearch / RabbitMQ / Filesystem / none Key methods — the methods that matter ``` --- ## Contents | § | Section | |---|---| | [0](#0-repository-root--what-is-actually-here) | Repository root — what is actually here (⚠ two backends) | | [1](#1-java-package-structure) | Java package structure + every backend file | | [2](#2-frontend-file-structure) | Frontend file structure + every frontend file | | [3](#3-file--database-matrix) | File → database matrix | | [4](#4-dead-code--unused-wiring) | Dead code and unused wiring | --- # 0. Repository root — what is actually here **Read this first.** The repository contains a *previous* implementation alongside the current one, and the top-level `README.md` describes the previous one. ``` E-Commere Order Management and Search Service-Java/ ├── backend-java/ ✅ THE RUNNING SERVICE — Spring Boot 3.2.5 / Java 21 │ ├── Dockerfile (multi-stage: maven:3.9.6-eclipse-temurin-21 → temurin:21-jre-alpine) │ └── src/main/java/… 76 Java files · src/test → 0 files │ ├── backend/ ⚠ LEGACY — Python FastAPI service (51 .py files) │ ├── app/api/… auth, health, orders, products, search, users │ ├── app/repositories/ postgres, mongo, elasticsearch │ └── requirements.txt ← NOT built by docker-compose.yml │ ├── frontend/ ✅ THE RUNNING FRONTEND — Vue 3 + Vite ├── docker-compose.yml every `build.context` is ./backend-java or ./frontend ├── tests/, pytest.ini ⚠ Python tests for the LEGACY backend (pytest not installed here) ├── README.md ⚠ documents FastAPI + Swagger /docs + python -m pytest ├── JAVA_MIGRATION_MAP.md ✅ describes the Python → Java migration ├── MIGRATION_REPORT.md ✅ same ├── update_all_images.py ⚠ out-of-band script (the image_url convention it defines is now implemented in SeedRunner — see SYSTEM_WORKFLOW.md Appendix B) ├── docs/ │ ├── MERIDIAN_COMPLETE_SYSTEM_KNOWLEDGE.md (9,983 lines — documents the PYTHON service) │ ├── FINAL_SYSTEM_AUDIT.md ✅ authoritative for the Java service │ ├── SYSTEM_ARCHITECTURE.md ✅ │ ├── SYSTEM_WORKFLOW.md ✅ │ └── FILE_BY_FILE_GUIDE.md ✅ (this file) │ ├── and/, Management/, Order/, Search/ │ └── ⚠ EMPTY directories — artifacts of an unquoted `mkdir` of this repo's own path │ ("E-Commere Order Management and Search Service-Java") ├── Knowledge bytes/Knowledge_Bytes.txt ⚠ stray note └── Service-Java/backend-java/src/… ⚠ empty directory tree (0 files) ``` **So, concretely:** * The **running** system is `backend-java/` + `frontend/` + `docker-compose.yml`. * `README.md` will lead you to FastAPI and Swagger at `:8000/docs` — the Java service has **no** Swagger UI; its API surface is the table in [`SYSTEM_ARCHITECTURE.md` §8](SYSTEM_ARCHITECTURE.md#8-api-map). * The legacy Python files are useful as a *reference* (the Java service was migrated from them) but nothing in `docker-compose.yml` executes them. --- # 1. Java package structure ``` com/meridian/backend/ entry point, seeder, /api/health ├── auth/ login/logout/current-user endpoints + authentication logic │ └── dto/ login/logout/user payloads ├── clients/*

*thin health-check wrappers over raw Mongo/ES/RabbitMQ clients ├── common/ │ ├── dto/ shared API payloads: error envelope, health, pagination, aggregations │ └── util/ UTC date helpers, BigDecimal money rounding ├── config/ security chain, CORS/Jackson, Mongo, RabbitMQ, Elasticsearch ├── core/config/ env-var config holder + manual JPA/Hikari setup ├── exception/ AppException hierarchy + the @RestControllerAdvice that renders them ├── images/ product image upload/delete endpoints + filesystem storage ├── messaging/ RabbitMQ publisher (order changed) + listener (index into ES) ├── orders/ order endpoints + order domain service (PostgreSQL) │ ├── dto/ order create/read/status payloads │ ├── entity/ JPA entities: Order, OrderItem, User │ └── repository/ Spring Data JPA repositories ├── products/ catalog endpoints + service (MongoDB) │ ├── document/ Mongo @Document mapping │ ├── dto/ product create/update/read/facets payloads │ └── repository/ Spring Data MongoDB repository ├── search/ admin order-search endpoint + ES query builder │ ├── dto/ search request/response + the ES order document │ └── repository/ direct Elasticsearch client class └── security/ JWT issue/validate, per-request filter, 401/403 handlers ``` --- ## Root package ### `MeridianApplication.java` **Purpose:** Spring Boot entry point; boots the app with Mongo auto-configuration excluded so `MongoConfig` owns Mongo wiring entirely. **Used by:** the JVM (`java -jar app.jar`). **Calls:** Spring runtime. **Database:** none directly. **Key methods:** `main(String[])`. Annotations: `@SpringBootApplication(exclude = MongoAutoConfiguration.class)`, `@EnableAsync`, `@EnableScheduling` (note: no `@Async`/`@Scheduled` methods exist anywhere). --- ### `SeedRunner.java` **Purpose:** the `seed`-profile seeder — wipes and refills users (PostgreSQL), products (MongoDB) and orders (PostgreSQL), applies the price-rename snapshot demo, then rebuilds the Elasticsearch index from scratch. **Used by:** Spring Boot startup, **only** when `--spring.profiles.active=seed`. **Calls:** `UserRepository`, `ProductRepository`, `OrderRepository`, `OrderSearchRepository`, `PasswordEncoder`, `DateUtils`, `MoneyUtils`. **Database:** PostgreSQL **+ MongoDB + Elasticsearch**. **Key methods:** - `run(ApplicationArguments)` — orchestrates `seedUsers → seedProducts → seedOrders → applySnapshotDemo → reindexSearch` - `seedUsers()` — 8 customers + `admin@example.com`, BCrypt-hashed via `passwordEncoder.encode` - `seedProducts()` — 27 `ProductDocument`s with tags/attributes/variants **and `image_url`** (`/products/<SKU>.svg`, `/products/default.svg` for `CHG-001`) - `seedOrders()` — 48 orders + 107 items; **two-phase** save (`TMP-…` then `ORD-%06d`) because `order_number` is `NOT NULL` + unique and the id is only known after insert - `applySnapshotDemo()` — renames `WM-001` to "Wireless Mouse Pro" and reprices it to ₹60, leaving historical order lines at "Wireless Mouse" / ₹50.16 - `reindexSearch()` — `searchRepository.recreateIndex()` **first** (order ids advance across seed runs, so stale docs would otherwise linger), then `bulkIndexOrders` - `buildSearchDocument(Order)` — maps an order to `ElasticsearchOrderDocument` (mirrors `OrderSyncListener`) --- ### `HealthController.java` **Purpose:** `GET /api/health` — pings all four dependencies and reports status + latency (200 `ok` / 503 `degraded`). **Used by:** browser and the Docker healthcheck. **Calls:** `MongoClientWrapper`, `ElasticsearchClientWrapper`, `RabbitMQClient`, `AppConfig`. **Database:** all four, read-only probes. **Key methods:** `health()`, `checkPostgres()` (`connection.isValid(2)`), `checkMongo()`, `checkElasticsearch()`, `checkRabbitMQ()`. --- ## `clients/` ### `MongoClientWrapper.java` **Purpose:** health façade — `{ping:1}` against the `admin` database using the **shared** `MongoClient`; deliberately never closes it. **Used by:** `HealthController`. **Calls:** `MongoClient` bean from `MongoConfig`. **Database:** MongoDB. **Key methods:** `ping()`, `getDatabase(String)`, `close()` (no-op). ### `ElasticsearchClientWrapper.java` **Purpose:** builds its **own** low-level ES client from `AppConfig` (separate from the `ElasticsearchConfig` data-access client) purely for health pings. **Used by:** `HealthController`. **Calls:** `AppConfig`. **Database:** Elasticsearch. **Key methods:** constructor, `ping()`, `getClient()`, `close()`. ### `RabbitMQClient.java` **Purpose:** builds a raw AMQP `ConnectionFactory` from `AppConfig.getRabbitmqUrl()` and opens a throw-away connection to prove reachability. **Used by:** `HealthController`. **Calls:** `AppConfig`. **Database:** RabbitMQ. **Key methods:** constructor, `ping()`. --- ## `common/dto/` | File | Purpose | Used by | DB | |---|---|---|---| | `ErrorResponse.java` | the `{"error": {…}}` envelope | `GlobalExceptionHandler`, both security handlers | none | | `ErrorBody.java` | `code` / `message` / `details` + nested `FieldError{field,message,type}` | `ErrorResponse`, exception handler | none | | `HealthResponse.java` | `status`, `environment`, `dependencies{}` | `HealthController` | none | | `HealthDependency.java` | one dependency's `status` + `latency_ms` | `HealthController` | none | | `SearchAggregations.java` | `revenue` (sum) and `statusCounts` (terms) | `OrderSearchRepository.parseAggregations`, `SearchService` | none | | `PaginationMeta.java` | `page/limit/total/pages` block | **nothing — dead code** (the response DTOs inline these fields) | none | --- ## `common/util/` ### `DateUtils.java` **Purpose:** canonical UTC time helpers so seed, services and converters agree on the zone. **Used by:** `SeedRunner`, `OrderService`, `ProductService`, `ImageService`, `OrderSyncListener`. **Database:** none. **Key methods:** `utcNow()` (`OffsetDateTime.now(UTC)`), `toIsoString()`, `startOfDay()`, `endOfDay()` (last three have no callers). ### `MoneyUtils.java` **Purpose:** BigDecimal money rounding to 2 dp HALF_UP and order-total summation. **Used by:** `OrderService` (`toMoney`, `calculateTotal`), `SeedRunner` (`toMoney`). **Database:** none. **Key methods:** `toMoney(Object)`, `calculateTotal(List)`, `calculateLineTotal()` (no external callers), `CENT`. --- ## `config/` ### `SecurityConfig.java` **Purpose:** the entire HTTP authorization policy and authentication machinery. **Used by:** Spring Security — runs on **every** HTTP request. **Calls:** `JwtAuthenticationFilter`, `CustomAuthenticationEntryPoint`, `CustomAccessDeniedHandler`, `UserRepository`, and the `CorsConfigurationSource` bean. **Database:** PostgreSQL (via `UserRepository`). **Key methods:** - `securityFilterChain(HttpSecurity)` — CSRF off, `STATELESS`, CORS from the injected source, the URL rules listed in [`SYSTEM_ARCHITECTURE.md` §8](SYSTEM_ARCHITECTURE.md#8-api-map), custom 401/403 handlers, and `addFilterBefore(jwtAuthenticationFilter, …)` - `userDetailsService()` — loads `User` **by email** into a Spring `User` with `roles(user.getRole())` → `ROLE_<role>` - `authenticationProvider()` → `DaoAuthenticationProvider` - `authenticationManager(AuthenticationConfiguration)` - `passwordEncoder()` → `BCryptPasswordEncoder` Annotations: `@EnableWebSecurity`, `@EnableMethodSecurity(prePostEnabled = true)`. > **Why `@EnableMethodSecurity` matters:** it is what makes `@PreAuthorize("hasRole('ADMIN')")` > effective — and those annotations are what actually protect product writes, because the URL > `permitAll` lines for `/api/products` match first. --- ### `WebConfig.java` **Purpose:** one CORS definition, the `/uploads/**` static mapping, and the app's `ObjectMapper`. **Used by:** Spring MVC and `SecurityConfig`. **Calls:** none of ours. **Database:** Filesystem (serves `app.uploads.dir`). **Key methods / beans:** - `corsConfigurationSource()` — **`@Bean @Primary`**; `/**` → allowed origins from `app.cors.allowed-origins`, methods `GET,POST,PUT,PATCH,DELETE,OPTIONS`, headers `*`, credentials on, `maxAge 3600`. **This bean is the CORS fix** — without it Spring resolved `SecurityConfig`'s dependency by type to `HandlerMappingIntrospector`, which returns `null` and makes preflights fail (see [`SYSTEM_WORKFLOW.md` Appendix C](SYSTEM_WORKFLOW.md#appendix-c--admin-login-issue-diagnosis)). - `corsFilter(CorsConfigurationSource)` — reuses the same source so the two cannot drift - `objectMapper()` — `JavaTimeModule`, `write-dates-as-timestamps: false`, `PropertyNamingStrategies.SNAKE_CASE` - `addResourceHandlers()` — `/uploads/**` → `file:${app.uploads.dir}/` --- ### `MongoConfig.java` **Purpose:** manual Mongo wiring (auto-config is excluded) including **`OffsetDateTime ⇄ BSON Date` conversion**. **Used by:** Spring; `MongoClient` feeds `MongoClientWrapper`, `MongoTemplate` feeds `ProductService`. **Database:** MongoDB. **Key methods / beans:** - `mongoCustomConversions()` — `MongoCustomConversions` with `OffsetDateTimeToDateConverter` (`@WritingConverter`) and `DateToOffsetDateTimeConverter` (`@ReadingConverter`) - `mongoClient()` — **`@Bean @Primary`**, `MongoClients.create(MongoClientSettings.builder() .applyConnectionString(new ConnectionString(mongoUri)).build())` — the single `MongoClient` - `mongoDatabaseFactory(MongoClient)` → `SimpleMongoClientDatabaseFactory` - `mongoTemplate(factory, conversions)` — passes the converter in explicitly so it is not dropped > **Do not "simplify" this file.** The sanctioned fix is the `MongoCustomConversions` pair — > a custom `CodecProvider`, a second `MongoClient`, or replacing `OffsetDateTime` fields with > `Instant` are all explicitly out of bounds. --- ### `RabbitMQConfig.java` **Purpose:** declares the sync topology and JSON conversion. **Used by:** Spring; `RabbitTemplate` → `SyncPublisher`, topology → `OrderSyncListener`. **Database:** RabbitMQ. **Key methods:** `syncExchange()` (`orders.sync.exchange`, durable), `syncQueue()` (durable `orders_sync`), `syncBinding(queue, exchange)` (routing key `orders.sync`), `jsonMessageConverter()` (`Jackson2JsonMessageConverter`), `rabbitTemplate(...)` (`mandatory = true`). Constants: `SYNC_EXCHANGE`, `SYNC_QUEUE`, `SYNC_ROUTING_KEY`. --- ### `ElasticsearchConfig.java` **Purpose:** builds the **primary** ES client stack used for indexing and searching, with JSR-310 date support. **Used by:** Spring; the `ElasticsearchClient` bean → `OrderSearchRepository`. **Database:** Elasticsearch. **Key methods:** `restClient()` (parses `spring.elasticsearch.uris`, optional basic auth, timeouts), `elasticsearchTransport(RestClient)` — `RestClientTransport` + `JacksonJsonpMapper` over an `ObjectMapper` **with `JavaTimeModule`** and ISO-8601 dates, `elasticsearchClient(ElasticsearchTransport)`, `parseTimeout(String)`. --- ## `core/config/` ### `AppConfig.java` **Purpose:** a flat `@Value` holder for every environment variable, mirroring the original Python service's env contract. **Used by:** `DatabaseConfig`, `HealthController`, `ElasticsearchClientWrapper`, `RabbitMQClient`. **Database:** none (config only). **Key methods:** `getPostgresJdbcUrl()`, `getRabbitmqUrl()` (falls back to `amqp://user:pass@host:port/`), `getCorsOriginList()`, `getSyncTaskName()`, plus ~35*

*explicit getters (`getElasticsearchIndex`, `getJwtSecretKey`, `isElasticsearchRefreshOnWrite`, …). ### `DatabaseConfig.java` **Purpose:** hand-rolled JPA stack — Hikari pool, `EntityManagerFactory`, transaction manager, repository scanning. **Used by:** Spring; consumed by all JPA repositories and services. **Database:** PostgreSQL. **Key methods:** `dataSource()` (`HikariDataSource` from `AppConfig`), `entityManagerFactory(DataSource)` (scans `orders.entity`, `PostgreSQLDialect`, `hbm2ddl.auto=validate`, UTC), `transactionManager(EntityManagerFactory)`. Annotations: `@EnableTransactionManagement`, `@EnableJpaRepositories(basePackages = "…orders.repository")`. --- ## `auth/` ### `AuthController.java` **Purpose:** HTTP surface for login/logout/current user under `/api/auth`. **Used by:** the Vue frontend (public). **Calls:** `AuthService`. **Database:** PostgreSQL (via the service). **Key methods:** `login(@Valid LoginRequest)` → `POST /api/auth/login`; `logout()` → `POST /api/auth/logout`; `me(@AuthenticationPrincipal User)` → `GET /api/auth/me`. ### `AuthService.java` **Purpose:** credential verification and JWT issuance. **Used by:** `AuthController`. **Calls:** `UserRepository`, `PasswordEncoder`, `JwtTokenProvider`, `NotFoundException`, `UnprocessableException`. **Database:** PostgreSQL. **Key methods:** - `authenticate(LoginRequest)` — `findByEmail` → reject empty hash → `passwordEncoder.matches` → `jwtTokenProvider.generateToken(id)` → `LoginResponse{access_token, "bearer", user}` - `getCurrentUser(User)` → `UserResponse` — **NPEs if `user` is null**, which is why unauthenticated `GET /api/auth/me` returns 500 - `mapToUserResponse(User)` (private) ### `auth/dto/LoginRequest.java` **Purpose:** login body. Fields: `email` (`@NotBlank @Email`), `password` (`@NotBlank @Size(1..128)`). **Note:** there is **no username field** — this is why `admin`/`admin` is rejected with 422. ### `auth/dto/LoginResponse.java` — `accessToken`, `tokenType` (default `"bearer"`), `user`. ### `auth/dto/LogoutResponse.java` — fixed `message: "Logged out successfully"` (stateless JWT; nothing is revoked). ### `auth/dto/UserResponse.java` — `id`, `name`, `email`, `role`. > A **second, duplicate** `UserResponse` exists as a nested class of `orders/dto/OrderResponse`. --- ## `exception/` | File | HTTP | Thrown when | |---|---|---| | `AppException.java` | base class | carries `code`, `statusCode`, `details` for every typed error | | `NotFoundException.java` | **404** | product/order/user not found | | `ConflictException.java` | **409** | duplicate SKU | | `UnprocessableException.java` | **422** | business rules: empty cart, quantity limit, inactive product, bad price, bad image, invalid credentials | | `DependencyUnavailableException.java` | **503** | Elasticsearch unavailable (`search_unavailable`) | | `OrderPersistenceException.java` | **500** | order write failed — *"No data was written"* | ### `GlobalExceptionHandler.java` **Purpose:** `@RestControllerAdvice` translating every exception into the `{"error":{code,message,details}}` envelope. **Used by:** Spring MVC, on every failing request. **Database:** none. **Key methods:** `handleAppException` (uses the exception's own status), `handleValidationException(MethodArgumentNotValidException)` → 422 with a `FieldError` list, `handleConstraintViolation` → 422, `handleHttpMessageNotReadable` → 400, `handleTypeMismatch` → 400, **`handleUnhandledException(Exception)` → 500 `internal_error`**. > ⚠️ That last catch-all is the cause of two known caveats: it converts Spring Security's > `AccessDeniedException` into **500 instead of 403/401**, and > `HttpRequestMethodNotSupportedException` into **500 instead of 405**. --- ## `security/` ### `JwtTokenProvider.java` **Purpose:** creates and validates the JWTs. **Used by:** `AuthService` (issue), `JwtAuthenticationFilter` (validate). **Database:** none. **Key methods:** - `generateToken(String userId)` — `subject(userId)`, `issuedAt`, `expiration`, **`signWith(secretKey)`** — note it does *not* read `app.jwt.algorithm`, so jjwt infers the algorithm from key length and real tokens are `{"alg":"HS384"}` - `validateToken(String)` — `parseSignedClaims`, `false` on any exception - `getUserIdFromToken(String)` — returns `claims.getSubject()` **Actual token contents:** `{"sub":"<user id>","iat":…,"exp":…}` — **no role, no email.** ### `JwtAuthenticationFilter.java` **Purpose:** per-request Bearer-token authentication (`extends OncePerRequestFilter`). **Used by:** the Spring Security filter chain (registered in `SecurityConfig`). **Calls:** `JwtTokenProvider`, `UserRepository`. **Database:** PostgreSQL (one lookup per authenticated request). **Key method:** `doFilterInternal(...)` — `Authorization: Bearer …` → validate → `getUserIdFromToken` → `userRepository.findById` → grant **`SimpleGrantedAuthority("ROLE_" + role)`** → set the `SecurityContextHolder` → continue. > The `ROLE_` prefix is precisely what makes `hasRole("ADMIN")` work. ### `CurrentUser.java` **Purpose:** annotation alias for `@AuthenticationPrincipal` — injects the authenticated `User` into a controller parameter. Used by `OrderController`, `ImageController`, `SearchController`. ### `CustomAuthenticationEntryPoint.java` **Purpose:** renders **401** `{"error":{"code":"unauthorized","message":"Could not validate credentials"}}` instead of the default redirect/blank. Used by `SecurityConfig`. ### `CustomAccessDeniedHandler.java` **Purpose:** renders **403** `{"error":{"code":"forbidden","message":"Admin access required"}}`. Used by `SecurityConfig`. --- ## `messaging/` ### `SyncPublisher.java` **Purpose:** publishes "order changed" messages so the search projection gets rebuilt. **Used by:** `OrderService.createOrder`, `OrderService.updateOrderStatus` (also injected, unused, into `SearchService`). **Calls:** `RabbitTemplate`. **Database:** RabbitMQ. **Key methods:** `publishOrderSync(Integer orderId, String reason)` — `convertAndSend` to `orders_sync`; **never throws** — returns `SyncInfo{enqueued:true, queue}` or `SyncInfo{enqueued:false, error}`. Nested: `OrderSyncMessage { orderId, reason }`. ### `OrderSyncListener.java` **Purpose:** consumes sync messages and re-indexes the order into Elasticsearch, stamping `search_indexed_at` back in PostgreSQL. **Used by:** the RabbitMQ broker (consumer side of `orders_sync`). **Calls:** `OrderRepository`, `OrderSearchRepository`, `DateUtils`. **Database:** RabbitMQ **(in) + PostgreSQL (read + write) + Elasticsearch (write)**. **Key methods:** - `handleOrderSync(OrderSyncMessage)` — annotated `@RabbitListener(queues = "${app.rabbitmq.sync-queue:orders_sync}")`; logs celery-style start/failure and rethrows on error (manual ack + retry policy apply) - `syncOrderDocument(Integer, String)` — `findByIdWithItemsAndUser` → `ensureIndex()` → `indexOrder()` → `orderRepository.updateSearchIndexedAt(...)` - `buildOrderDocument(Order)` — same mapping as `SeedRunner.buildSearchDocument` > Runs in **both** the `backend` and `java-worker` containers → competing consumers. --- ## `orders/` ### `OrderController.java` **Purpose:** HTTP surface for order create/read/status under `/api/orders`. **Used by:** the Vue frontend (authenticated). **Calls:** `OrderService`, order DTOs. **Database:** PostgreSQL (via the service). **Key methods:** - `createOrder(OrderCreate, @CurrentUser User)` → `POST /api/orders`, returns **201** - `getOrder(@PathVariable Integer, @CurrentUser User)` → `GET /api/orders/{id}`; **403** if a non-admin asks for someone else's order, **404** if it does not exist - `updateStatus(orderId, StatusUpdateRequest)` → `PATCH /api/orders/{id}/status`, `@PreAuthorize("hasRole('ADMIN')")` ### `OrderService.java` **Purpose:** order domain logic — cart resolution/pricing against the Mongo catalog, PostgreSQL persistence with two-phase order-number generation, and sync publication. **Used by:** `OrderController`. **Database:** PostgreSQL **+ MongoDB (pricing) + RabbitMQ (publish)**. **Key methods:** - `@Transactional createOrder(...)` — forces `userId` from the principal → `resolveCart()` → save with `TMP-…` → re-save as `ORD-%06d` → save each `OrderItem` → on any non-`AppException` throw `OrderPersistenceException` (rollback) → **finally** `publishOrderSync(id, "order_created")` - `getOrder(Integer)` — `findByIdWithItemsAndUser`, 404 if missing - `@Transactional updateOrderStatus(Integer, String)` — flips status, stamps `updatedAt`, publishes `"status_update"`; an unchanged status returns `SyncInfo{queue:"in-process"}` - `resolveCart(List<OrderItemCreate>)` (private) — merges duplicates, enforces `MAX_LINE_QUANTITY = 100`, `productRepository.findByIds`, rejects missing/inactive/invalid price, computes the total - mapping helpers + inner types `ResolvedLine`, `ResolvedCart` - `listUsers()` — **dead code**: no controller maps `GET /api/users` ### `orders/entity/Order.java` **Table:** `orders` (indexes on unique `order_number`, `status`, `order_date`, `user_id`, `(user_id, order_date)`). **Fields:** `id`, `orderNumber`, `user` (`@ManyToOne LAZY`), `orderDate`, `status` (default `"PENDING"`), `totalAmount`, `createdAt`, `updatedAt`, `searchIndexedAt`, `items` (`@OneToMany(cascade=ALL, orphanRemoval=true)`, `@OrderBy("id ASC")`). **Key methods:** `getCustomer()` (alias for `getUser()`), `recalculateTotal()` (2 dp HALF_UP). ### `orders/entity/OrderItem.java` **Table:** `order_items` (indexes `order_id`, `product_id`). **Fields:** `id`, `order` (`@ManyToOne LAZY`), `productId` (**the Mongo `_id`**, not the SKU), `title` (**snapshot**), `quantity`, `unitPrice` (**snapshot**). **Key methods:** `getLineTotal()` = `unitPrice × quantity`, 2 dp HALF_UP. ### `orders/entity/User.java` **Table:** `users` (unique index on `email`). **Fields:** `id`, `name`, `email`, `passwordHash`, `role`, `createdAt`, `orders` (`@OneToMany`). **Key methods:** none beyond accessors. Also consumed by `SecurityConfig.userDetailsService`, `AuthService` and `JwtAuthenticationFilter`. ### `orders/repository/OrderRepository.java` **Purpose:** JPA access to `orders`. **Database:** PostgreSQL. **Key methods:** `findByIdWithItemsAndUser(id)` (`JOIN FETCH` items + user — used everywhere a full order is needed), `updateSearchIndexedAt(orderId, indexedAt)` (`@Modifying`), plus `countAllOrders`, `countAllOrderItems`, `findOrdersAfterId`, `getStatusCounts` (**no callers**). ### `orders/repository/OrderItemRepository.java` **Purpose:** `JpaRepository<OrderItem, Integer>` — the `save` used by `createOrder`. `findByOrderId` has **no callers**. **Database:** PostgreSQL. ### `orders/repository/UserRepository.java` **Purpose:** `JpaRepository<User, Integer>`. **Key methods:** `findByEmail(String)` (login + `userDetailsService`), `findAllOrderByIdAsc()` (only used by the dead `listUsers`). **Used by:** `AuthService`, `JwtAuthenticationFilter`, `SecurityConfig`, `OrderService`, `SeedRunner`. **Database:** PostgreSQL. ### `orders/dto/` | File | Purpose | |---|---| | `OrderCreate.java` | request body: `userId` (`@Min(1)`, **overwritten server-side**), `items` (`@NotNull @Size(1..50)`) | | `OrderItemCreate.java` | line: `productId` (`@NotBlank @Size(1..64)`), `quantity` (`@NotNull @Min(1) @Max(100)`) | | `OrderResponse.java` | read model incl. nested `customer{id,name,email,role}` and `items[]` | | `OrderCreateResponse.java` | order fields **+*

*`sync`** (`SyncInfo`), returned with 201 | | `StatusUpdateResponse.java` | order fields **+ `sync`**, returned by the PATCH | | `StatusUpdateRequest.java` | single field `status` (`@NotNull`) | | `OrderItemResponse.java` | `id`, `productId`, `title`, `quantity`, `unitPrice`, `lineTotal` | | `SyncInfo.java` | `{enqueued, taskId, queue, error}` — the RabbitMQ outcome block | --- ## `products/` ### `ProductController.java` **Purpose:** HTTP surface for the catalog under `/api/products`. **Used by:** the Vue frontend (reads public, writes ADMIN). **Calls:** `ProductService`. **Database:** MongoDB (via the service). **Key methods:** - `listProducts(category, tags, q, visibility, page, limit, sort)` → `GET /api/products` (`visibility` regex `active|inactive|all`, `sort` regex `newest|title_asc|price_asc|price_desc`, `page ≤ 10000`, `limit ≤ 100`) - `productFacets()` → `GET /api/products/facets` - `getProduct(productId)` → `GET /api/products/{id}` - `createProduct` → 201, `updateProduct` → PATCH, `deleteProduct` → DELETE — **all three carry `@PreAuthorize("hasRole('ADMIN')")`** ### `ProductService.java` **Purpose:** catalog CRUD plus filtering/faceting directly against MongoDB via `MongoTemplate`. **Used by:** `ProductController`, `ImageController`. **Database:** MongoDB. **Key methods:** - `listProducts(...)` — builds `Criteria` (active flag, category exact, `tags $all`, `q` as a case-insensitive `$regex` over title/description/sku/tags), sort mapping, `mongoTemplate.count` + `find` → `ProductListResponse` with computed `pages` - `listFacets()` — `Aggregation.facet()`: category counts + top-40 tag counts (active only) - `getProduct(String)` — 404 `product_not_found` - `createProduct(ProductCreate)` — duplicate-SKU check → `ConflictException` 409 - `updateProduct(String, ProductUpdate)` — partial `Update` via `mongoTemplate.updateFirst`, SKU uniqueness via `findBySkuAndIdNot`, price > 0 rule, stamps `updatedAt` - `deleteProduct(String)` — **soft delete**: `active = false` - `updateImageUrl` / `removeImageUrl` — keep `imageUrl` in sync after an upload - `normalizeTags`, `toResponse`, variant converters; `ensureIndexes()` is an **empty uncalled stub** ### `products/document/ProductDocument.java` **Collection:** `products`. **Fields:** `_id`, `sku` (unique index `uniq_products_sku`), `title`, `description`, `price`, `category` (`ix_products_category`), `tags[]`, `attributes{}`, `variants[{sku,color,stock}]`, `active` (`ix_products_active`), **`image_url`**, `created_at`, `updated_at`. Carries Lombok annotations **and** hand-written getters/setters **and** hand-written builders. **Used by:** `ProductRepository`, `ProductService`, `ImageService`, `OrderService`, `SeedRunner`. ### `products/repository/ProductRepository.java` **Purpose:** `MongoRepository<ProductDocument, String>`. **Key methods:** `findBySku(String)`, `findBySkuAndIdNot(sku, id)`, `findByIds(List<String>)` (`@Query { '_id': { $in: ?0 } }` — used by `OrderService.resolveCart`). Unused: `findByActiveTrue`, `findByCategoryAndActiveTrue`, `findByTagsInAndActiveTrue`, `countAll`. ### `products/dto/` | File | Purpose | |---|---| | `ProductCreate.java` | admin create body: `sku` (`@NotBlank @Size(2..64)` + pattern), `title` (2..200), `price` (`0.01..1_000_000`), `category`, `tags`, `attributes`, `variants`, `active`, `imageUrl` | | `ProductUpdate.java` | same fields, all-nullable partial update | | `VariantDTO.java` | `{sku, color, stock}` with validation | | `ProductResponse.java` | full read model incl. `image_url`, `updated_at`, `created_at`, `id` | | `ProductListResponse.java` | `{items, page, limit, total, pages}` | | `ProductFacetsResponse.java` | `{categories:[{value,count}], tags:[{value,count}]}` | --- ## `images/` ### `ImageController.java` **Purpose:** multipart image endpoints on the `/api/products` path. **Used by:** the Vue frontend (`ProductFormModal`), ADMIN. **Calls:** `ImageService`, `ProductService`. **Database:** Filesystem + MongoDB. **Key methods:** `uploadProductImage(productId, @RequestPart("file") MultipartFile, @CurrentUser)` → `POST /api/products/{id}/image` (consumes `multipart/form-data`); `removeProductImage(...)` → `DELETE /api/products/{id}/image`. Both carry `@PreAuthorize("hasRole('ADMIN')")`. ### `ImageService.java` **Purpose:** validates, writes and deletes product image files, keeping `imageUrl` in sync. **Used by:** `ImageController`. **Database:** Filesystem + MongoDB. **Key methods:** - `uploadImage(productId, file)` — 404 check → `validateImageFile` → `generateSafeFilename` → `Files.createDirectories` + `Files.copy` into `/app/uploads/products` → delete a replaced `/uploads/...` file → set `imageUrl = "/uploads/products/<uuid><ext>"` → save - `removeImage(productId)` — delete the file, null `imageUrl`, save - `validateImageFile` (private) — allow-list `image/jpeg|png|webp|svg+xml` → `invalid_image_format`; 5 MB cap → `image_too_large` - `generateSafeFilename` (private) — UUID hex + allowed extension, fallback `.webp` --- ## `search/` ### `SearchController.java` **Purpose:** the single admin-only search endpoint. **Used by:** `AdminSearchView.vue`. **Calls:** `SearchService`. **Database:** Elasticsearch (via the service). **Key method:** `searchOrders(@Valid SearchRequest, @CurrentUser User)` → `POST /api/search/orders`, `@PreAuthorize("hasRole('ADMIN')")`. ### `SearchService.java` **Purpose:** translates admin criteria into an ES bool query, and the ES response into `SearchResponse`. **Used by:** `SearchController`. **Database:** Elasticsearch. **Key methods:** - `searchOrders(SearchRequest)` — `from/size`, sort `-order_date,-order_id`; a 400 from ES → `AppException(search_query_invalid, 400)`, anything else → `DependencyUnavailableException(search_unavailable, 503)` - `buildQuery(SearchRequest)` — bool `must` (omni text) + filters (`status` terms, `order_date` range with `dateTo + 1 day`, `total_amount` range); **falls back to `matchAll`** - `buildOmniQuery(String)` — **AND across terms** + phrase `multi_match` on `customer.name^3`/`customer.email` + uppercased `order_number` prefix, `minimumShouldMatch(1)` - `termQuery(String)` — flat `multi_match` (`order_number^3`, `customer.name^3`), **nested** `multi_match` (`items.title^2`, `items.product_id`), and an `order_number` prefix - `convertResponse`, `convertToSearchResult` (private) ### `search/repository/OrderSearchRepository.java` **Purpose:** direct Elasticsearch client class (not Spring Data) — index lifecycle, indexing, search with aggregations, response parsing. **Used by:** `SearchService`, `OrderSyncListener`, `SeedRunner`. **Database:** Elasticsearch. **Key methods:** - `ensureIndex()` — creates `orders` (1 shard / 0 replicas) with the full mapping including **`nested` items** and `search_as_you_type` customer/title fields - `deleteIndex()`, `recreateIndex()` - `indexOrder(doc)` — `_id = order_id`, `Refresh.True` when configured - `bulkIndexOrders(List)` — used by the seeder - `search(Query, from, size, sort)` — `trackTotalHits`, field sorts, plus aggregations `revenue` (`sum total_amount`) and `status_counts` (`terms status`, size 10) - `parseAggregations` — seeds PENDING/PROCESSING/SHIPPED to `0` so KPI cards never go blank - `countDocuments()` — **no callers** ### `search/dto/` | File | Purpose | |---|---| | `SearchRequest.java` | `query (≤200)`, `status[]`, `dateFrom/dateTo (LocalDate)`, `minPrice/maxPrice`, `page (1..10000)`, `limit (1..100)` | | `SearchResponse.java` | `{results, total, page, limit, pages, took_ms, aggregations}` | | `SearchResult.java` | one hit + nested `SearchCustomer` / `SearchItem` | | `ElasticsearchOrderDocument.java` | the JSON written to / read from ES; `@JsonProperty` forces snake_case (`order_id`, `order_number`, `order_date`, `updated_at`, `total_amount`, `product_id`, `unit_price`, `line_total`) | --- # 2. Frontend file structure ``` frontend/src/ ├── main.js bootstrap: createApp → Pinia → router → mount ├── App.vue <router-view/> + <ToastHost/>; calls authStore.init() ├── api/ axios wrapper modules (one per resource) ├── router/index.js route table, guards, titles, scrollBehavior ├── stores/ auth · cart · toast ├── layouts/ StorefrontLayout · AdminLayout ├── views/ 6 route-level pages ├── components/ │ ├── common/ 19 design-system primitives │ ├── storefront/ 8 store-page components │ ├── checkout/ 3 checkout sub-components │ ├── auth/ login form │ └── admin/ 12 admin components ├── styles/ tokens.css · utilities.css └── utils/ image · currency · dates · status · validation · debounce ``` ## Entry & root | File | Purpose | Used by | |---|---|---| | `main.js` | creates the app, installs `createPinia()` + router, imports both stylesheets, mounts `#app` | Vite | | `App.vue` | renders `<router-view/>` + `<ToastHost/>`; runs `authStore.init()` on setup | `main.js`, smoke tests | ## `api/` — API modules ### `api/client.js` **Purpose:** the single axios instance and all error handling. **Imported by every api module.** **Key exports and behaviour:** - `resolveBaseUrl()` — `VITE_API_BASE_URL`: `""` → relative (proxy/nginx); unset → `http://localhost:8000`; trailing slashes stripped. Exported as `API_BASE_URL`. - `axios.create({ baseURL, timeout: 20000, paramsSerializer: { indexes: null } })` — the serializer emits repeated keys (`?tags=a&tags=b`) - **request interceptor** — reads `localStorage['meridian.auth.token']` and sets `Authorization: Bearer …` - **response interceptor** — unwraps `response.data`, and converts every failure via `toApiError()` into an `ApiError { code, status, details }`: envelope `message` → `code`; no response at all → `network_error` *"Cannot reach the API server…"*; otherwise `client_error` - `fieldErrorsFrom(error)` — maps a 422's `details.fields[]` to `{field: message}` - `messageFrom(error)` — best human-readable message | Module | Export | Endpoint | |---|---|---| | `api/products.js` | `listProducts` | `GET /api/products` | | | `getProductFacets` | `GET /api/products/facets` | | | `getProduct` | `GET /api/products/{id}` | | | `createProduct` | `POST /api/products` | | | `updateProduct` | `PATCH /api/products/{id}` | | | `deleteProduct` | `DELETE /api/products/{id}` (soft) | | | `uploadProductImage` | `POST /api/products/{id}/image` (FormData) | | | `removeProductImage` | `DELETE /api/products/{id}/image` | | `api/orders.js` | `createOrder` | `POST /api/orders` — sends `{items:[{product_id, quantity}]}`, **no prices** | | | `getOrder` | `GET /api/orders/{id}` | | | `updateOrderStatus` | `PATCH /api/orders/{id}/status` | | `api/search.js` | `searchOrders` | `POST /api/search/orders` | | `api/users.js` | `listUsers` | `GET /api/users` — **imported by nothing (dead)** | ## `router/index.js` **Purpose:** route table, guards, `document.title`, `scrollBehavior`. | Path | Name | Component | meta | |---|---|---|---| | `/login` | `login` | `views/LoginView.vue` | `{ title:'Sign in', guest:true }` | | `/` | — | `layouts/StorefrontLayout.vue` (eager) | — | | `/` → `""` | `storefront` | `views/StorefrontView.vue` | `{ title:'Storefront' }` | | `/checkout` | `checkout` | `views/CheckoutView.vue` | `{ title:'Checkout', requiresAuth:true }` | | `/admin` | — | `layouts/AdminLayout.vue` (eager) | `{ requiresAuth:true, requiresAdmin:true }` | | `/admin` → `""` | — | redirect → `admin-search` | — | | `/admin/search` | `admin-search` | `views/AdminSearchView.vue` | `{ title:'Search dashboard' }` | | `/admin/orders/:id` | `order-details` |

*`views/OrderDetailsView.vue` | `{ title:'Order details' }` | | `/admin/catalog` | `catalog-admin` | `views/CatalogAdminView.vue` | `{ title:'Catalog admin' }` | | `/:pathMatch(.*)*` | — | redirect → `storefront` | — | **Guard logic (verbatim):** ```js router.beforeEach(async (to, from, next) => { const authStore = useAuthStore() if (!authStore.loading && authStore.token && !authStore.user) await authStore.fetchMe() const requiresAuth = to.matched.some(r => r.meta.requiresAuth) const requiresAdmin = to.matched.some(r => r.meta.requiresAdmin) const isGuest = to.matched.some(r => r.meta.guest) if (requiresAuth && !authStore.isAuthenticated) return next({ name: 'login', query: { redirect: to.fullPath } }) if (requiresAdmin && !authStore.isAdmin) return next({ name: 'storefront' }) if (isGuest && authStore.isAuthenticated) return next({ name: 'storefront' }) next() }) ``` `scrollBehavior` returns `savedPosition`, or `false` when only `q/category/sort/page` changed (so smooth-scroll to `#products-section` is not cancelled), else `{top:0}`. ## `stores/` ### `stores/auth.js` **State:** `token`, `user`, `loading`, `error`. **Persistence:** `localStorage['meridian.auth.token']` and `['meridian.auth.user']`. **Getters:** `isAuthenticated` (`!!token && !!user`), `isAdmin` (`user?.role === 'ADMIN'`), `userName`, `userEmail`, `userId`. **Actions:** - `login(email, password)` → `POST /api/auth/login` → store both keys → `router.push(query.redirect || '/')`; on error sets `error` and rethrows - `logout()` → `POST /api/auth/logout` (errors ignored) → clear both keys → `/login` - `fetchMe()` → no-op without a token; `GET /api/auth/me` → update `user`; any failure clears the session - `init()` — called by `App.vue`; revalidates a restored session **Used by:** `App.vue`, router guards, both layouts, `LoginForm`, `CheckoutView`. > **`isAdmin` reads the `user` object from localStorage — it never decodes the JWT.** ### `stores/cart.js` **State:** `items: { [productId]: { id, sku, title, price, category, image_url, quantity } }` — a snapshot taken at add time (display only; the server re-prices). **Getters:** `lines`, `isEmpty`, `itemCount`, `subtotal`. **Actions:** `add(product, qty=1)` (merges; returns `null` if no `product.id`), `setQuantity` (clamps 1..100 via `MIN_QTY`/`MAX_QTY`), `increment`, `decrement`, `remove`, `clear()`. **Persistence: none** — no `localStorage` write anywhere, so a reload empties the cart. ### `stores/toast.js` **State:** `toasts[]` (capped at 5), a module-level id counter. **Actions:** `push(type, message, timeout)`, `success`, `error` (6500 ms), `info`, `warning`, `dismiss(id)`. Consumed by `ToastHost.vue`. ## `layouts/` ### `layouts/StorefrontLayout.vue` **Purpose:** the public shell — announcement bar, sticky header with brand/nav/search, account menu, cart button with count badge, mobile panel, `<main><router-view/>`, 3-column footer, and the `<CartDrawer>`. **Logic:** URL-driven search debounced 260 ms writing `?q=`; `/` and `Ctrl/Cmd+K` focus search, `Esc` closes; `goCategory()` writes `?category=`; the **Admin** nav link renders only when `authStore.isAdmin`; `logout()` → `authStore.logout()`. **Children:** `AppIcon`, `CartDrawer`. ### `layouts/AdminLayout.vue` **Purpose:** the admin shell — fixed sidebar (brand, "Search dashboard", "Catalog admin", data-source note, "View storefront", user name/role + Logout), mobile overlay, sticky topbar showing `route.meta.title`, content `<router-view/>`. **Children:** `AppIcon`, `BaseButton`. ## `views/` | View | Route | Responsibility | Children | API / stores | |---|---|---|---|---| | `StorefrontView.vue` | `/` | hero, category rail, tag marquee, active-filter chips, filters, grid, pagination. **The URL query (`q`,`category`,`sort`,`page`) is the single source of truth** | `HeroSection`, `CategorySection`, `TagMarquee`, `ProductFilters`, `ProductGrid`, `AppPagination` | `listProducts`, `getProductFacets`; cart, toast | | `CheckoutView.vue` | `/checkout` | **live re-price** of every line via `Promise.allSettled(getProduct)`, marks unavailable rows, computes subtotal, `POST /api/orders`, clears cart, shows success | `CheckoutLineItems`, `OrderSummaryCard`, `OrderSuccessPanel`, `EmptyState` | `getProduct`, `createOrder`; cart, auth, toast | | `LoginView.vue` | `/login` | branded card; on `@success` → `route.query.redirect \|\| '/'` | `LoginForm` | — | | `AdminSearchView.vue` | `/admin/search` | KPIs, filters, count + "took X ms", error/empty/results states, desktop table + mobile cards, pagination; 350 ms debounce; filter changes reset to page 1 | `SearchKpis`, `SearchFilters`, `ResultsTable`, `ResultsCardList`, `AppPagination` | `searchOrders` | | `OrderDetailsView.vue` | `/admin/orders/:id` | skeleton/404/error/loaded states; "Source of truth: PostgreSQL" badge, sync indicator, status control, Customer/Totals/Line-items cards | `OrderStatusControl`, `SearchSyncIndicator`, `AppCard`×3 | `getOrder`, `updateOrderStatus` | | `CatalogAdminView.vue` | `/admin/catalog` | search box, category/visibility selects, New product, result meta, table/card states, pagination, create/edit modal, deactivate confirm | `CatalogTable`, `CatalogCardList`, `ProductFormModal`, `ConfirmDialog` | `listProducts`, `getProductFacets`, `updateProduct`, `deleteProduct` | ## `components/storefront/` | File | Props | Emits | Notes | |---|---|---|---| | `HeroSection.vue` | — | — | fetches its own `listProducts({limit:5, sort:'newest'})`; featured + 2 previews; add-to-cart | | `CategorySection.vue` | `categories[]`, `loading` | — | rail with counts; selection writes `?category=` | | `TagMarquee.vue` | `tags[]`, `clickable`, `speed` | — | duplicated-track marquee; static under `prefers-reduced-motion` | | `TagPill.vue` | `tag`, `clickable` | — | click → `?q=<tag>` | | `ProductFilters.vue` | `category`, `sort`, `selectedTags`, `categoryFacets`, `tagFacets`, … | `update:category`, `update:sort`, `reset` | sort options `newest/title_asc/price_asc/price_desc`; **tag props declared but never rendered** | | `ProductGrid.vue` | `items[]`, `loading`, `error` | `add`, `clear`, `retry` | 4-column grid with bespoke skeleton; **imports `LoadingSkeleton` but never uses it** | | `ProductCard.vue` | `product` | `add(product)` | uses `ProductThumb`; "Add to cart" flips to "Added to cart" for 1400 ms | | `CartDrawer.vue` | `open` | `close`, `checkout` | qty 1..100, remove, subtotal, checkout | ## `components/checkout/` | File | Props | Emits | Notes | |---|---|---|---| | `CheckoutLineItems.vue` | `rows[]`, `loading` | — | unit price + line total + subtotal; "No longer available" badge | | `OrderSummaryCard.vue` | `itemCount`, `subtotal`, `customer`, `placing`, `disabled` | `place` | **`CheckoutView` never passes `customer`** → renders "No customer selected" | | `OrderSuccessPanel.vue` | `order` | `continue` | order number/total/status + the RabbitMQ sync alert (`order.sync.enqueued`); "View order details" routes to the **admin-only** `order-details` | ## `components/auth/` ### `LoginForm.vue` **Props:** `redirect` (declared, **never read** — the store performs the redirect). **Emits:** `success`. **Behaviour:** client-side `validateEmail()` / `validatePassword()`, then `authStore.login(email, password)`, showing `authStore.error`. **Renders the demo hint:** `john.doe@example.com / john, admin@example.com / admin`. **Children:** `FormField`, `BaseInput`, `BaseButton`. ## `components/admin/` | File | Props | Emits | Used by | |---|---|---|---| | `SearchKpis.vue` | `aggregations`, `total`, `loading` | — | 5 `KpiCard`s: revenue, filtered, pending, processing, shipped (passes `tone="accent"`, for which no style is defined → neutral) | | `SearchFilters.vue` | `query`, `statuses[]`, `dateFrom/To`, `minPrice/maxPrice`, `activeCount` | `update:*`, `reset` | status chips sorted `PENDING, PROCESSING, SHIPPED`, date/price ranges | | `ResultsTable.vue` | `items[]`, `loading` | `select(id)` | desktop results; `order_number` is a `RouterLink` to `order-details` | | `ResultsCardList.vue` | `items[]`, `loading` | `select(id)` | mobile results | | `OrderStatusControl.vue` | `status`, `pending`, `updating` | `change(status)` | `BaseSelect` from `statusOptions()` | | `SearchSyncIndicator.vue` | `indexedAt` | — | "Indexed" / "Not yet indexed" pill | | `CatalogTable.vue` | `items[]`, `loading` | `edit`, `toggle` | image, SKU, title, price, category, tags, variants, active badge | | `CatalogCardList.vue` | `items[]` | `edit`, `toggle` | mobile catalog | | `ProductFormModal.vue` | `open`, `product` (null ⇒ create) | `close`, `saved(product, action)` | validates via `utils/validation`, uploads image (5 MB), maps 422 via `fieldErrorsFrom()` onto fields | | `TagsInput.vue` | `modelValue[]`, `placeholder`, `invalid`, `max` | `update:modelValue` | Enter/comma commits, Backspace removes, lowercased + deduped | | `AttributesEditor.vue` | `modelValue{}` | `update:modelValue` | key/value rows → object, coerces `"5"`/`"true"` | | `VariantsEditor.vue` | `modelValue[]` | `update:modelValue` | rows `{sku, color, stock}` | ## `components/common/` — 19 primitives `AppIcon` (~50 inline SVG names) · `BaseButton` · `BaseInput` · `BaseSelect` · `BaseTextarea` · `FormField` (provides the field id) · `AppAlert` · `AppBadge` · `StatusBadge` · `AppCard` · `AppModal` (Teleport + focus trap) · `AppDrawer` · `AppPagination` · `ConfirmDialog` · `EmptyState` · `KpiCard` · `LoadingSkeleton` · **`ProductThumb`** · `ToastHost`. ### `ProductThumb.vue` — the image component ```imageUrl = resolveImageUrl(product.image_url) <div v-else class="thumb__placeholder">monogram + deterministic pattern``` `null`, a load error, or a failed load all fall back to the placeholder. **Sizes:** `sm|md|lg`. **Used by:** `ProductCard`, `CartDrawer`, `HeroSection`, `CheckoutLineItems`, `CatalogTable`, `CatalogCardList`. ## `utils/` | File | Exports | Used by | |---|---|---| | `image.js` | `resolveImageUrl(value)` | `ProductThumb`, `ProductFormModal` | | `currency.js` | `formatCurrency`, `formatNumber`, `parseAmount` (INR `en-IN`) | ~16 components | | `dates.js` | `toUtcDate`, `formatDate/Time/DateTime`, `toDateInput`, `isValidDate` | order + catalog views | | `status.js` | `ORDER_STATUSES = ['PENDING','PROCESSING','SHIPPED']`, `isOrderStatus`, `statusLabel/Tone/Options` | `StatusBadge`, `SearchFilters`, `OrderStatusControl` | | `validation.js` | `SKU_PATTERN`, `validateSku/Title/Price/Category/Variant(s)/AttributeKeys`, `coerceAttributeValue` | product form | | `debounce.js` | `debounce(fn, wait)` with `.cancel()` | storefront search, admin search, catalog | ### `resolveImageUrl` — exact logic ```js 1. non-string / empty → null → caller renders placeholder 2. /(^\|[a-z][a-z0-9+.-]*:)\/\//i or /^(data|blob):/i → returned untouched (https://…, //cdn/…, previews) 3. startsWith('/uploads/') && API_BASE_URL → API_BASE_URL + path (uploads are served by the BACKEND) 4. anything else → returned as-is (bundled /products/<SKU>.svg assets) ``` ## Styles & build | File | Purpose | |---|---| | `styles/tokens.css` | `:root` design tokens (`--color-primary:#FFAC1C`, teal, surfaces, radii, shadows, spacing) + reset/base | | `styles/utilities.css` | legacy alias tokens + shared utilities (`.tabular`, `.truncate`, `.sr-only`, `.only-mobile`) | | `vite.config.js` | `host 0.0.0.0`, `port 5173`, proxies `/api` and `/uploads` → `VITE_PROXY_TARGET \|\| localhost:8000`; vitest config (`jsdom`, url `http://localhost:5173/` so CORS accepts test requests, `tests/**/*.spec.js`) | | `package.json` | scripts `dev`/`build`/`preview`/`test`/`test:watch`; deps `axios pinia vue vue-router`;*

*devDeps `@vitejs/plugin-vue @vue/test-utils jsdom vite vitest` | | `nginx.conf` (prod) | proxies `/api/` and `/uploads/` → `http://backend:8000`, caches `/assets/` 7 days, SPA fallback | **Tests (`frontend/tests/`, 5 files, 28 tests — all passing):** | File | Covers | |---|---| | `smoke.spec.js` | full-app mounts against the **live API**: storefront cards, admin KPIs + ES results, empty checkout, live order details, catalog table. Logs in as `admin@example.com/admin` and seeds `localStorage` | | `cart.spec.js` | add/merge/clamp/subtotal/remove/clear | | `components.spec.js` | `BaseButton`, `StatusBadge`, `KpiCard` | | `status.spec.js` | `utils/status` + `utils/dates` (UTC handling) | | `currency.spec.js` | INR formatting, `formatNumber`, `parseAmount` | --- # 3. File → database matrix Legend: **W** = writes · **R** = reads · **P** = pings only · **.** = untouched. | File | PostgreSQL | MongoDB | Elasticsearch | RabbitMQ | Filesystem | |---|---|---|---|---|---| | **`MeridianApplication.java`** | · | · | · | · | · | | **`SeedRunner.java`** | W | W | W | · | · | | **`HealthController.java`** | R | P | P | P | · | | `clients/MongoClientWrapper` | · | P | · | · | · | | `clients/ElasticsearchClientWrapper` | · | · | P | · | · | | `clients/RabbitMQClient` | · | · | · | P | · | | `common/util/DateUtils` | · | · | · | · | · | | `common/util/MoneyUtils` | · | · | · | · | · | | `config/SecurityConfig` | R | · | · | · | · | | **`config/WebConfig`** | · | · | · | · | **W** (resource handler) | | **`config/MongoConfig`** | · | **connection** | · | · | · | | `config/RabbitMQConfig` | · | · | · | **topology** | · | | `config/ElasticsearchConfig` | · | · | **connection** | · | · | | `core/config/AppConfig` | · | · | · | · | · | | `core/config/DatabaseConfig` | **connection** | · | · | · | · | | **`auth/AuthController`** | via service | · | · | · | · | | **`auth/AuthService`** | **R/W** | · | · | · | · | | `exception/GlobalExceptionHandler` | · | · | · | · | · | · | | **`security/JwtAuthenticationFilter`** | **R** | · | · | · | · | | `security/JwtTokenProvider` | · | · | · | · | · | | `security/CustomAuthenticationEntryPoint` | · | · | · | · | · | | `security/CustomAccessDeniedHandler` | · | · | · | · | · | | **`messaging/SyncPublisher`** | · | · | · | **W** | · | | **`messaging/OrderSyncListener`** | **R/W** | · | **W** | **R** | · | | **`orders/OrderController`** | via service | · | · | · | · | | **`orders/OrderService`** | **W** | **R** (pricing) | · | **W** | · | | `orders/entity/Order` | **table** | · | · | · | · | | `orders/entity/OrderItem` | **table** | · | · | · | · | | `orders/entity/User` | **table** | · | · | · | · | | `orders/repository/OrderRepository` | **W** | · | · | · | · | | `orders/repository/OrderItemRepository` | **W** | · | · | · | · | | `orders/repository/UserRepository` | **R** | · | · | · | · | | **`products/ProductController`** | · | via service | · | · | · | | **`products/ProductService`** | · | **W** | · | · | · | | `products/document/ProductDocument` | · | **collection** | · | · | · | | `products/repository/ProductRepository` | · | **R/W** | · | · | · | · | | **`images/ImageController`** | · | via service | · | · | via service | | **`images/ImageService`** | · | **W** | · | · | **W** | | **`search/SearchController`** | · | · | via service | · | · | · | | **`search/SearchService`** | · | · | · | **R** | · | · | | **`search/repository/OrderSearchRepository`** | · | · | **W/R** | · | · | · | | `search/dto/ElasticsearchOrderDocument` | · | · | payload | · | · | · | | — | | | | | | **FRONTEND** | | | | | | | `api/orders.js` | **R/W** (via HTTP) | · | · | · | · | · | | `api/products.js` | · | **R/W** | · | · | **W** (upload) | | `api/search.js` | · | · | · | **R** | · | · | · | | `api/client.js` | · | · | · | · | · | reads `localStorage` | | `stores/auth.js` | · | · | · | · | · | `localStorage` | | `stores/cart.js` | · | · | · | · | · | in-memory only | | `utils/image.js` | · | · | · | · | · | resolves URLs | | everything else | · | · | · | · | · | --- # 4. Dead code and unused wiring Discovered while inventorying. Harmless, but useful to know so you do not chase it. **Backend** * `orders/../users/` — `backend-java/src/main/java/com/meridian/backend/users/` is an **empty directory**: no package, no classes (the `users` concern lives in `orders/entity/User`). * `OrderService.listUsers()` — no controller maps `GET /api/users`, yet `SecurityConfig` `permitAll`s that path. * `ProductService.ensureIndexes()` — empty, never called. * `PaginationMeta` — never referenced (response DTOs inline the fields). * Unused repository methods: `OrderRepository.countAllOrders / countAllOrderItems / findOrdersAfterId / getStatusCounts`, `OrderItemRepository.findByOrderId`, `ProductRepository.findByActiveTrue / findByCategoryAndActiveTrue / findByTagsInAndActiveTrue / countAll`, `OrderSearchRepository.countDocuments`. * Unused helpers: `DateUtils.toIsoString / startOfDay / endOfDay`, `MoneyUtils.calculateLineTotal / CENT`. * `SyncPublisher` injected into `SearchService` but never used; `ErrorResponse` imported in four controllers without use. * `@EnableAsync` / `@EnableScheduling` with no `@Async` / `@Scheduled` methods. * `app.jwt.algorithm` declared in `application.yml` but never read. * Duplicate `UserResponse` (top-level in `auth/dto` and nested in `orders/dto/OrderResponse`). **Frontend** * `api/users.js` — imported by nothing. * `ProductGrid.vue` imports `LoadingSkeleton.vue` but uses its own skeleton markup. * `ProductFilters.vue` declares tag props/emits the parent never passes and the template never renders. * `LoginForm.vue`'s `redirect` prop is never read. * `OrderSummaryCard.vue` accepts a `customer` prop `CheckoutView` never passes. * `SearchKpis.vue` passes `tone="accent"`, but `KpiCard.vue` defines no `.kpi--accent` style. --- *See also:* [`SYSTEM_ARCHITECTURE.md`](SYSTEM_ARCHITECTURE.md) · [`SYSTEM_WORKFLOW.md`](SYSTEM_WORKFLOW.md) · [`FINAL_SYSTEM_AUDIT.md`](FINAL_SYSTEM_AUDIT.md)*

Response: Here is the updated, project-tailored prompt template configured specifically for **Meridian**. It bakes in the exact multi-database boundaries, request pipelines, code-path nuances, and critical caveats from your documentation.

Customized Prompt: Meridian Knowledge Bytes Generator

```
You are acting as a patient, precise senior engineer walking a new teammate through the Meridian e-commerc
```

CONTEXT & SYSTEM LAWS (Keep these strictly in mind):

1. **The Active Stack:** The running service is the Java 21 / Spring Boot 3.2.5 backend (`backend-java/`) and Vue 3 + Vite frontend (`frontend/`). Ignore the legacy Python FastAPI implementation (`backend/`) and its tests.
2. **Polyglot Persistence Boundaries:**
   - **MongoDB (7):** Product catalog authority (nested documents: tags, variants, attributes). Queried live during checkout.
   - **PostgreSQL (16):** ACID transactional source of truth for users, orders, and order lines.
   - **RabbitMQ (3.13):** Asynchronous sync message bus (`orders.sync.exchange` -> `orders_sync`).
   - **Elasticsearch (8.15):** Disposable search projection for admin order search, status/date facets, and KPI aggregations (`orders` index).
   - **Worker Architecture:** `java-worker` runs the identical `app.jar` as `backend` without a published port (they act as competing consumers on `orders_sync`).
3. **Core Design Conventions:**
   - **Order Snapshot Semantics:** An order freezes product `title` and `unit_price` in `order_items` at purchase time; it never rewrites history when catalog prices drift.
   - **Two-Phase Order Creation:** PostgreSQL requires a provisional `TMP-` order number on first save, followed by an immediate update to `ORD-%06d` in the same transaction.
   - **Stateless Auth:** JWT subject holds `userId`. Roles are not in the token; `JwtAuthenticationFilter` fetches the user from PostgreSQL per request and maps `ROLE_`.
   - **Two Image Pipelines:** Bundled artwork lives in `frontend/public/products/*.svg` (served by Vite/Nginx). Admin uploads live in `/app/uploads` (served by Spring Boot `/uploads/**`).

RULES FOR GENERATING KNOWLEDGE BYTES:

1. **Single Responsibility:** Each byte covers ONE isolated concept, architectural boundary, class method, data model, or workflow step.
2. **Progressive Order:**
    - Start with high-level purpose/context (e.g., Polyglot rationale, Container topology).
    - Move to structural foundations (e.g., Spring Security filter chain, Data schemas, Snapshot semantics).
    - Move to concrete execution flows (e.g., Checkout transaction, RabbitMQ sync, ES Query construction).
    - Finish with gotchas, edge cases, and known caveats (e.g., CORS preflight mechanics, competing consumers, shadowed URL security matchers).
3. **Plain Language First:** Explain the purpose and the "why" in plain, conversational English before showing code or config.
4. **Concrete Snippets Only:** Include only the exact, relevant snippet (Java, JavaScript/Vue, SQL, or JSON) directly tied to the point. No bloated file dumps.
5. **No Blind Syntax Transliteration:** Explain what the mechanism accomplishes and why it was designed that way (e.g., why MongoDB handles catalog attributes without migrations, or why range queries in ES need the typed client builder).
6. **Brevity:** Keep each byte readable in under 45 seconds (~100 to 180 words).

OUTPUT FORMAT (Repeat for each byte):

Byte [N]: [Short, Descriptive Title]

Builds on: [Byte # or "None — Starting point"] **System / Database:** [e.g., PostgreSQL | MongoDB | Elasticsearch | RabbitMQ | Spring Boot | Vue 3 | Multi-system]

In plain terms: [1–3 sentences explaining the core responsibility, why it exists, and how it behaves in everyday language.]

The code / design:

```
[Concise snippet of code, configuration, mapping, or query]
```

Under the hood & why it matters:

- **Mechanism:** [Brief explanation of what the snippet executes and the direct callers/callees.]
- **Design rationale / Gotcha:** [Key trade-off, architectural invariant, or caveat (e.g., snapshot isolation, transactional rollback, competing consumer throughput).]

Suggested Byte Roadmap to Generate First

When using this prompt, you can instruct the model to produce bytes across this natural sequence:

1. **System Foundation & Topology:**
    - Byte 1: Polyglot Persistence Division of Labor
    - Byte 2: Docker Topology & Internal DNS (`localhost` vs Service Names)
    - Byte 3: Competing Consumers (`backend` vs `java-worker`)
2. **Data & Snapshot Guarantees:**
    - Byte 4: The MongoDB Product Document & Dynamic Attributes
    - Byte 5: Order Snapshot Semantics (Why History Never Rewrites)
    - Byte 6: Two-Phase Order Number Generation (`TMP-` to `ORD-%06d`)
3. **Auth & Security Pipeline:**
    - Byte 7: Stateless JWT with Database-Backed Role Hydration
    - Byte 8: CORS Preflight & Why `CorsConfigurationSource` Matters
    - Byte 9: Method Security vs Shadowed URL Matchers (`@PreAuthorize`)
4. **Checkout & Async Sync Workflow:**
    - Byte 10: Live Cart Re-Pricing at Checkout
    - Byte 11: The `@Transactional` Order Boundary & Decoupled AMQP Publish
    - Byte 12: RabbitMQ Worker Consumption & Idempotent ES Indexing
5. **Search & Asset Delivery:**
    - Byte 13: Elasticsearch Omni-Query (Term-AND vs OR semantics)
    - Byte 14: Nested Item Matching in Elasticsearch

- Byte 15: Dual Image Architecture (Bundled SVG vs Volume Uploads)

User prompt: okay use this prompt on the files have uploaded above and create knowledge bytes for the same You are acting as a patient, precise senior engineer walking a new teammate through the Meridian e-commerce service codebase. Your job is to break down the system architecture, workflows, and source files into a series of small, self-contained explanations called "KNOWLEDGE BYTES."

Response: Welcome to the Meridian codebase walkthrough. Below is a progressive breakdown of the system architecture, data models, workflows, and core operational mechanics structured as small, bite-sized knowledge bytes.

```
<svg viewBox="0 0 680 180" width="100%" role="img" aria-label="Meridian Polyglot Persistence Architecture"
  <defs>
    <marker id="arrow" viewBox="0 0 10 10" refX="9" refY="5" markerWidth="6" markerHeight="6" orient="auto
      <path d="M0,0 L10,5 L0,10 z" fill="currentColor" />
    </marker>
  </defs>
  <!-- Client -->
  <rect x="20" y="65" width="100" height="50" rx="8" fill="none" stroke="currentColor" stroke-width="1.5"
  <text x="70" y="88" text-anchor="middle" font-size="12" font-weight="bold" fill="currentColor">Vue 3 Cli
  <text x="70" y="103" text-anchor="middle" font-size="10" fill="currentColor">:5173 / Vite</text>

  <!-- Spring Boot Backend -->
  <rect x="180" y="65" width="130" height="50" rx="8" fill="none" stroke="currentColor" stroke-width="1.5"
  <text x="245" y="88" text-anchor="middle" font-size="12" font-weight="bold" fill="currentColor">Spring B
  <text x="245" y="103" text-anchor="middle" font-size="10" fill="currentColor">Java 21 (:8000)</text>

  <line x1="120" y1="90" x2="178" y2="90" stroke="currentColor" stroke-width="1.5" marker-end="url(#arrow)

  <!-- MongoDB -->
  <rect x="370" y="15" width="110" height="42" rx="6" fill="none" stroke="currentColor" stroke-width="1.5"
  <text x="425" y="34" text-anchor="middle" font-size="11" font-weight="bold" fill="currentColor">MongoDB
  <text x="425" y="47" text-anchor="middle" font-size="9" fill="currentColor">Catalog Authority</text>

  <!-- PostgreSQL -->
  <rect x="370" y="70" width="110" height="42" rx="6" fill="none" stroke="currentColor" stroke-width="1.5"
  <text x="425" y="89" text-anchor="middle" font-size="11" font-weight="bold" fill="currentColor">PostgreS
  <text x="425" y="102" text-anchor="middle" font-size="9" fill="currentColor">ACID Order Truth</text>

  <!-- RabbitMQ -->
  <rect x="370" y="125" width="110" height="42" rx="6" fill="none" stroke="currentColor" stroke-width="1.5
  <text x="425" y="144" text-anchor="middle" font-size="11" font-weight="bold" fill="currentColor">RabbitM
  <text x="425" y="157" text-anchor="middle" font-size="9" fill="currentColor">orders_sync queue</text>

  <!-- Java Worker -->
  <rect x="540" y="125" width="115" height="42" rx="6" fill="none" stroke="currentColor" stroke-width="1.5
  <text x="597" y="144" text-anchor="middle" font-size="11" font-weight="bold" fill="currentColor">Java Wo
  <text x="597" y="157" text-anchor="middle" font-size="9" fill="currentColor">Competing Consumer</text>

  <!-- Elasticsearch -->
  <rect x="540" y="40" width="115" height="42" rx="6" fill="none" stroke="currentColor" stroke-width="1.5"
  <text x="597" y="59" text-anchor="middle" font-size="11" font-weight="bold" fill="currentColor">Elastics
  <text x="597" y="72" text-anchor="middle" font-size="9" fill="currentColor">Search Projection</text>

  <!-- Connections from Backend -->
  <path d="M310,80 L340,36 L368,36" fill="none" stroke="currentColor" stroke-width="1.2" marker-end="url(#
  <line x1="310" y1="91" x2="368" y2="91" stroke="currentColor" stroke-width="1.2" marker-end="url(#arrow)
  <path d="M310,102 L340,146 L368,146" fill="none" stroke="currentColor" stroke-width="1.2" marker-end="ur

  <!-- Async Sync to Worker to ES -->
  <line x1="480" y1="146" x2="538" y2="146" stroke="currentColor" stroke-width="1.2" marker-end="url(#arro
  <line x1="597" y1="125" x2="597" y2="84" stroke="currentColor" stroke-width="1.2" marker-end="url(#arrow
</svg>
```

Byte 1: Polyglot Persistence & Division of Authority

Builds on: None — Starting point

System / Database: Architecture Overview

In plain terms:

Meridian does not store all its data in one relational database. Instead, each data store owns a dedicated concern shaped precisely for its query patterns: MongoDB stores flexible product documents, PostgreSQL guarantees strict transactional orders, RabbitMQ decouples write workloads, and Elasticsearch powers instant faceted search.

The code / design:

```
# Four specialized data stores define clear authorities:
mongodb:        # Collection: products (Nested variants, attributes, tags)
postgres:       # Tables: users, orders, order_items (ACID transactional writes)
rabbitmq:       # Exchange: orders.sync.exchange -> Queue: orders_sync
elasticsearch:  # Index: orders (Disposable read-optimized projection)
```

Under the hood & why it matters:

- **Mechanism:** PostgreSQL is the single source of truth for all placed orders. MongoDB acts as the catalog authority. Elasticsearch owns *no* authoritative truth—its `orders` index can be dropped and regenerated from PostgreSQL via `SeedRunner.recreateIndex()` at any time without data loss.
- **Design rationale / Gotcha:** Isolating search indexing via message queues ensures that placing an order never blocks on cluster latency or indexing spikes. If Elasticsearch goes offline, customer checkout remains unaffected; only `POST /api/search/orders` reports a `503 search_unavailable`.

Byte 2: Active Service vs Legacy Python Implementation

Builds on: Byte 1

System / Database: Build & Deployment Topology

In plain terms:

The repository houses two different backend codebases side-by-side, but only one is built and run. The active system is Spring Boot 3.2.5 running on Java 21; the legacy Python FastAPI implementation at `backend/` is not targeted by `docker-compose.yml`.

The code / design:

```
# docker-compose.yml targets backend-java only:
services:
  backend:
    build:
      context: ./backend-java
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
  java-worker:
    build:
      context: ./backend-java
      dockerfile: Dockerfile
    # shares the identical app.jar image; publishes no ports
```

Under the hood & why it matters:

- **Mechanism:** The top-level `README.md` and root `tests/` still point to FastAPI and Swagger documentation at `:8000/docs`. The active Java application has no Swagger UI; its API endpoints are mapped explicitly across Spring controllers.
- **Design rationale / Gotcha:** Do not run `pytest` to test the running system—use `vitest run` inside `frontend/` and review the 76 Java source files inside `backend-java/src/main/java`.

Byte 3: The Competing Consumers Model

Builds on: Byte 1, Byte 2

System / Database: Spring Boot / RabbitMQ / Docker

In plain terms:

The `java-worker` container is not a separate application codebase—it is built from the exact same Java artifact as the primary backend. Both instances bind to RabbitMQ as competing consumers on the same message queue to share indexing workloads.

The code / design:

```
@Component
public class OrderSyncListener {
    // Both backend and java-worker start this listener on container boot
    @RabbitListener(queues = "${app.rabbitmq.sync-queue:orders_sync}")
    public void handleOrderSync(OrderSyncMessage message) {
        syncOrderDocument(message.getOrderId(), message.getReason());
    }
}
```

Under the hood & why it matters:

- **Mechanism:** Because both services boot `ENTRYPOINT ["java", "-jar", "app.jar"]` with no distinguishing Spring profile, Spring AMQP automatically registers listeners on both. They split message ingestion across two active nodes.
- **Design rationale / Gotcha:** Because `indexOrder` in `OrderSearchRepository` writes documents using `_id = order_id`, duplicate reads or competing writes are completely idempotent overwrites.

Byte 4: Catalog Representation in MongoDB

Builds on: Byte 1

System / Database: MongoDB / Spring Data Mongo

In plain terms:

Products contain flexible facets like colors, stock counts, tags, and hardware specifications. Storing them as unified JSON-like documents in MongoDB eliminates relational SQL joins when rendering a product card or calculating catalog facets.

The code / design:

```
@Document(collection = "products")
public class ProductDocument {
    @Id private String id;
    @Indexed(unique = true) private String sku;
    private Double price;
    private List<String> tags;
    private Map<String, Object> attributes; // Arbitrary dynamic specifications
    private List<VariantDTO> variants;      // [{sku, color, stock}]
    private Boolean active;
    @Field("image_url") private String imageUrl;
}
```

Under the hood & why it matters:

- **Mechanism:** `ProductService.listProducts()` maps incoming query filters directly to MongoDB `Criteria`. Dynamic attributes like keyboard switch types or mouse DPI reside within `attributes` without requiring database schema migrations.
- **Design rationale / Gotcha:** `MongoAutoConfiguration` is deliberately excluded in `MeridianApplication.java`. Custom type converters (`OffsetDateTimeToDateConverter` and `DateToOffsetDateTimeConverter`) inside `MongoConfig` prevent BSON serialization mismatches.

Byte 5: Order Snapshot Semantics

Builds on: Byte 1, Byte 4

System / Database: PostgreSQL / Database Design

In plain terms:

When an order is created, the system copies the product's title and price into PostgreSQL line items as immutable values. Changes to the live catalog in MongoDB never modify historical orders or alter customer receipts.

The code / design:

```
-- order_items holds frozen copies, NOT live references to products
CREATE TABLE order_items (
    id          SERIAL PRIMARY KEY,
    order_id    INTEGER REFERENCES orders(id) ON DELETE CASCADE,
    product_id  VARCHAR(64) NOT NULL, -- The Mongo _id
    title       VARCHAR(255) NOT NULL, -- Frozen snapshot
    quantity    INTEGER NOT NULL,
    unit_price  NUMERIC(12, 2) NOT NULL -- Frozen snapshot
);
```

Under the hood & why it matters:

- **Mechanism:** In `SeedRunner.applySnapshotDemo()`, the live catalog item `WM-001` is renamed to "Wireless Mouse Pro" and repriced to ₹60.00. Historical lines placed earlier remain recorded as "Wireless Mouse" at ₹50.16.
- **Design rationale / Gotcha:** Financial and audit logs must remain unchangeable. `order_items` stores MongoDB's internal string `_id` rather than human-readable SKU strings, preventing accidental relational cascades if a catalog entry is deactivated.

Byte 6: Two-Phase Order Number Generation

Builds on: Byte 5

System / Database: Spring Data JPA / PostgreSQL

In plain terms:

PostgreSQL requires `order_number` to be non-null and unique, but the user-facing sequential order number depends on the auto-generated database primary key. The application solves this chicken-and-egg problem with a two-phase write inside a single transaction.

The code / design:

```
// OrderService.java (inside @Transactional createOrder)
Order order = new Order();
order.setOrderNumber("TMP-" + UUID.randomUUID()); // Phase 1: Temporary unique string
orderRepository.save(order);                      // Generates auto-increment ID

order.setOrderNumber(String.format("ORD-%06d", order.getId())); // Phase 2: Formatted identifier
orderRepository.save(order);                      // Persisted before transaction commit
```

Under the hood & why it matters:

- **Mechanism:** Both `save()` calls occur within the boundary of an active `@Transactional` context. Hibernate executes an `INSERT` followed by an `UPDATE`, releasing the finalized order number (e.g., `ORD-000293`) atomically upon transaction commit.
- **Design rationale / Gotcha:** Using a temporary UUID prevents constraint violations without requiring custom external sequence tables or risking race conditions across parallel order submissions.

Byte 7: Stateless JWT Authentication with Dynamic Role Hydration

Builds on: Byte 1

System / Database: Spring Security / PostgreSQL

In plain terms:

The JWT token issued at login contains only the user ID—it carries no role or email claims. On every single API request, the security filter extracts the user ID and loads the user from PostgreSQL to determine the active authority.

The code / design:

```
// JwtAuthenticationFilter.java
String userId = jwtTokenProvider.getUserIdFromToken(token);
User user = userRepository.findById(Integer.valueOf(userId)).orElse(null);

if (user != null) {
    // Prefix "ROLE_" is mandatory for Spring hasRole(...) checks
    var authority = new SimpleGrantedAuthority("ROLE_" + user.getRole());
    var auth = new UsernamePasswordAuthenticationToken(user, null, List.of(authority));
    SecurityContextHolder.getContext().setAuthentication(auth);
}
```

Under the hood & why it matters:

- **Mechanism:** Appending `ROLE_` to the string stored in the database (e.g., `ADMIN` becomes `ROLE_ADMIN`) allows Spring Security's `hasRole("ADMIN")` expressions to evaluate correctly.
- **Design rationale / Gotcha:** Because roles are hydrated dynamically from PostgreSQL on each request, revoking an administrator's permissions in the database takes effect immediately, despite the JWT token remaining valid on the client.

Byte 8: The CORS Preflight & Primary Configuration Trap

Builds on: Byte 7

System / Database: Spring MVC / Security Configuration

In plain terms:

Browsers issue an HTTP `OPTIONS` preflight request before sending JSON payloads with custom headers. If Spring does not have an explicit, primary `CorsConfigurationSource` bean, the preflight request fails with a 403 Forbidden even if the credentials and route logic are fully valid.

The code / design:

```
// WebConfig.java
@Bean
@Primary
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(appConfig.getCorsOriginList()); // http://localhost:5173
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

Under the hood & why it matters:

- **Mechanism:** Declaring `@Primary` ensures Spring resolves this bean ahead of internal fallback inspectors like `HandlerMappingIntrospector`, which defaults to returning `null` and causes `DefaultCorsProcessor` to reject preflight calls.
- **Design rationale / Gotcha:** When debugging browser errors where simple `GET` requests succeed via `curl` but `POST /api/auth/login` fails in the browser UI, inspect the network panel for failing `OPTIONS` requests.

Byte 9: Shadowed URL Matchers & Method Security

Builds on: Byte 7, Byte 8

System / Database: Spring Security / Exception Handling

In plain terms:

Spring Security evaluates URL authorization rules from top to bottom and stops at the first match. Because public product reads are declared first using broad wildcards, admin write endpoints rely on method-level annotations to enforce access controls.

The code / design:

```
// SecurityConfig.java: /api/products/** is marked permitAll early:
.requestMatchers("/api/products/**").permitAll()
.requestMatchers(HttpMethod.POST, "/api/products").hasRole("ADMIN") // Shadowed!

// ProductController.java: Method security protects the operation:
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<ProductResponse> createProduct(@Valid @RequestBody ProductCreate dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(productService.createProduct(dto));
}
```

Under the hood & why it matters:

- **Mechanism:** Enabling `@EnableMethodSecurity(prePostEnabled = true)` enforces role checks via runtime reflection proxies on controller methods, bypassing the shadowed HTTP path matchers.
- **Design rationale / Gotcha:** When an unauthorized user attempts an admin write, `@PreAuthorize` throws an `AccessDeniedException`. In this codebase, `GlobalExceptionHandler` catches generic exceptions and renders an HTTP 500 status rather than standard 401/403 errors, though the write is still aborted.

Byte 10: Live Cart Re-Pricing at Checkout

Builds on: Byte 4, Byte 5

System / Database: Vue 3 / Pinia / MongoDB

In plain terms:

The client-side cart stores only an in-memory preview of items and estimated prices. When navigating to checkout and submitting an order, the server completely ignores client-supplied prices and re-reads authoritative pricing directly from MongoDB.

The code / design:

```
// CheckoutView.vue (frontend re-verification on mount)
await Promise.allSettled(cart.lines.map(line => getProduct(line.id)))

// OrderService.java (backend authoritative enforcement)
private ResolvedCart resolveCart(List<OrderItemCreate> items) {
    List<String> ids = items.stream().map(OrderItemCreate::getProductId).toList();
    Map<String, ProductDocument> catalog = productRepository.findByIds(ids);
```

```
    // Unit price is extracted directly from the database document:
    BigDecimal price = MoneyUtils.toMoney(catalog.get(item.getProductId()).getPrice());
}
```

Under the hood & why it matters:

- **Mechanism:** `POST /api/orders` accepts only an array of `{product_id, quantity}` pairs. `OrderService.resolveCart()` checks whether each item is active, enforces a maximum quantity limit of 100 per line, and calculates totals with `BigDecimal` rounding.
- **Design rationale / Gotcha:** Client-side storage tampering cannot alter checkout prices. Additionally, because the Pinia cart does not write to `localStorage`, browser page reloads clear the transient cart state.

Byte 11: Transactional Commit & Out-of-Band AMQP Publishing

Builds on: Byte 3, Byte 5, Byte 6

System / Database: Spring Data JPA / RabbitMQ

In plain terms:

Placing an order executes within a strict database transaction. Message publishing to RabbitMQ happens only after the database write succeeds. If message queuing fails, the placed order is preserved rather than rolled back.

The code / design:

```
// OrderService.java
@Transactional
public OrderCreateResponse createOrder(OrderCreate payload, User currentUser) {
    Order order = persistOrderAndItems(payload, currentUser); // DB commit boundary

    SyncInfo syncInfo;
    try {
        syncInfo = syncPublisher.publishOrderSync(order.getId(), "order_created");
    } catch (Exception ex) {
        syncInfo = SyncInfo.failed("RabbitMQ unavailable");
    }
    return new OrderCreateResponse(order, syncInfo);
}
```

Under the hood & why it matters:

- **Mechanism:** `SyncPublisher.publishOrderSync` sends a compact JSON payload containing `{order_id, reason}` to the topic exchange `orders.sync.exchange`. `SyncInfo` captures enqueue status without bubbling up connection errors.
- **Design rationale / Gotcha:** Placing an order must never abort due to downstream search ingestion issues. The frontend displays the confirmation immediately and inspects `sync.enqueued` to determine whether search indexing is currently pending.

Byte 12: Asynchronous Elasticsearch Projection & Timestamp Stamping

Builds on: Byte 1, Byte 3, Byte 11

System / Database: Java Worker / Elasticsearch / PostgreSQL

In plain terms:

The background worker listens for order change messages, reads the authoritative order record with all items and customer data from PostgreSQL, transforms it into an optimized search document, and writes it to Elasticsearch before stamping an indexed timestamp in PostgreSQL.

The code / design:

```
// OrderSyncListener.java
private void syncOrderDocument(Integer orderId, String reason) {
    // Fetch JOIN prevents N+1 query overhead
    Order order = orderRepository.findByIdWithItemsAndUser(orderId).orElseThrow();
    ElasticsearchOrderDocument doc = buildOrderDocument(order);

    orderSearchRepository.indexOrder(doc); // Idempotent index write
    orderRepository.updateSearchIndexedAt(orderId, DateUtils.utcNow());
}
```

Under the hood & why it matters:

- **Mechanism:** `OrderRepository.updateSearchIndexedAt()` marks the completion of the projection in PostgreSQL. The frontend admin order detail view checks `search_indexed_at` to render the "Indexed" badge.

- **Design rationale / Gotcha:** The listener is configured with manual ACKs, 5 retries with exponential backoff, and `default-requeue-rejected: false`. Poison messages that cannot be parsed are dropped to prevent endless retry loops.

Byte 13: Elasticsearch Omni-Query Construction

Builds on: Byte 1, Byte 12

System / Database: Elasticsearch / Java Client

In plain terms:

Admin order searching uses an omni-search bar that searches across customer names, emails, order numbers, and item titles simultaneously. The query builder splits search text into terms and combines them using AND logic rather than OR logic so additional search terms narrow the results.

The code / design:

```
// SearchService.java
BoolQuery.Builder boolBuilder = new BoolQuery.Builder();
for (String term : terms) {
    // Every separate term must match at least one candidate field
    boolBuilder.must(termQuery(term));
}
// Boost whole phrases for customer matches
boolBuilder.should(phraseMatch("customer.name", text, 3.0f));
```

Under the hood & why it matters:

- **Mechanism:** If terms were joined via OR semantics, typing `Wendy Wireless` would match every order placed by Wendy plus every order containing any wireless accessory (matching 48 out of 48 seeded records). Enforcing term-level AND logic narrows the matches down to 6.
- **Design rationale / Gotcha:** Range filters for date and price must be constructed using the Java client's typed builder (`.field("total_amount").gte(JsonData.of(...))`). Passing pre-formatted JSON range strings causes Elasticsearch to reject the query with a 400 Bad Request.

Byte 14: Nested Item Mapping in Elasticsearch

Builds on: Byte 12, Byte 13

System / Database: Elasticsearch Indexing

In plain terms:

Each order contains multiple purchased items. In Elasticsearch, lines are mapped as `nested` objects rather than standard object arrays, preventing cross-object field pollution during multi-field queries.

The code / design:

```
// Elasticsearch mapping snippet for orders index:
{
  "mappings": {
    "properties": {
      "items": {
        "type": "nested",
        "properties": {
          "product_id": { "type": "keyword" },
          "title":      { "type": "search_as_you_type" },
          "unit_price": { "type": "double" },
          "quantity":   { "type": "integer" }
        }
      }
    }
  }
}
```

Under the hood & why it matters:

- **Mechanism:** Standard arrays flatten properties into parallel lists (e.g., `items.title = ["Mouse", "Keyboard"]`, `items.unit_price = [20, 100]`), which causes false matches when querying for a "Mouse" with a price of "100". Mapping as `nested` indexes each array element as an independent hidden Lucene document.
- **Design rationale / Gotcha:** Querying items inside `SearchService` requires wrapping item conditions inside a `NestedQuery` block targeting path `items`.

Byte 15: Dual Image Pipeline Mechanics

Builds on: Byte 4

System / Database: Vue 3 / Spring MVC / Docker Volumes

In plain terms:

Product artwork is delivered through two distinct mechanisms: pre-packaged catalog drawings are bundled and served directly by the frontend web server, while custom images uploaded through the admin panel are saved to disk and served by the Spring Boot backend.

The code / design:

```
// frontend/src/utils/image.js
export function resolveImageUrl(value) {
  if (!value) return null; // Triggers SVG monogram placeholder
  if (value.startsWith('/uploads/')) {
    // Path B: Uploaded by admin, served by backend container
    return `${API_BASE_URL}${value}`;
  }
  // Path A: Bundled artwork (e.g. /products/WM-001.svg), served by frontend
  return value;
}
```

Under the hood & why it matters:

- **Mechanism:** Path A assets reside under `frontend/public/products/` and bypass backend requests entirely. Path B routes multipart uploads through `ImageController` into `/app/uploads/products`, served via Spring's resource handler registered in `WebConfig`.
- **Design rationale / Gotcha:** If `image_url` is missing or null in MongoDB, `ProductThumb.vue` falls back to a procedural client-side placeholder. The seed runner explicitly assigns relative SVG paths (e.g., `/products/WM-001.svg`) to prevent null image fields.
